

Help on package seaborn:

NAME

```
seaborn - # Import seaborn objects
```

PACKAGE CONTENTS

```
_base  
_compat  
_core (package)  
_docstrings  
_marks (package)  
_statistics  
_stats (package)  
_testing  
algorithms  
axisgrid  
categorical  
cm  
colors (package)  
distributions  
external (package)  
matrix  
miscplot  
objects  
palettes  
rcmod  
regression
```

relational

utils

widgets

DATA

```
crayons = {'Almond': '#EFDECD', 'Antique Brass': '#CD9575', 'Apricot':...
```

```
xkcd_rgb = {'acid green': '#8ffe09', 'adobe': '#bd6c48', 'algae': '#54...
```

VERSION

0.13.2

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn__init__.py

Help on module seaborn.algorithms in seaborn:

NAME

seaborn.algorithms - Algorithms to support fitting routines in seaborn plotting functions.

FUNCTIONS

bootstrap(*args, **kwargs)

Resample one or more arrays with replacement and store aggregate values.

Positional arguments are a sequence of arrays to bootstrap along the first

axis and pass to a summary function.

Keyword arguments:

`n_boot : int, default=10000`

Number of iterations

`axis : int, default=None`

Will pass axis to ``func`` as a keyword argument.

`units : array, default=None`

Array of sampling unit IDs. When used the bootstrap resamples units and then observations within units instead of individual datapoints.

`func : string or callable, default="mean"`

Function to call on the args that are passed in. If string, uses as name of function in the numpy namespace. If nans are present in the data, will try to use nan-aware version of named function.

`seed : Generator | SeedSequence | RandomState | int | None`

Seed for the random number generator; useful if you want reproducible resamples.

Returns

`boot_dist: array`

array of bootstrapped statistic values

FILE

`c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\algorithms.py`

Help on module `seaborn.axisgrid` in `seaborn`:

NAME

`seaborn.axisgrid`

CLASSES

`Grid(_BaseGrid)`

`FacetGrid`

`PairGrid`

`_BaseGrid(builtins.object)`

`JointGrid`

```
class FacetGrid(Grid)
```

```
| FacetGrid(  
|     data,  
|     *,  
|     row=None,  
|     col=None,  
|     hue=None,  
|     col_wrap=None,  
|     sharex=True,  
|     sharey=True,  
|     height=3,  
|     aspect=1,  
|     palette=None,  
|     row_order=None,  
|     col_order=None,  
|     hue_order=None,
```

```
|     hue_kws=None,

|     dropna=False,

|     legend_out=True,

|     despine=True,

|     margin_titles=False,

|     xlim=None,

|     ylim=None,

|     subplot_kws=None,

|     gridspec_kws=None

| )

|

| Multi-plot grid for plotting conditional relationships.

|

| Method resolution order:

|     FacetGrid

|     Grid

|     _BaseGrid

|     builtins.object

|

| Methods defined here:

|

| __init__(

|     self,

|     data,

|     *,

|     row=None,

|     col=None,
```

```
|     hue=None,  
|  
|     col_wrap=None,  
|  
|     sharex=True,  
|  
|     sharey=True,  
|  
|     height=3,  
|  
|     aspect=1,  
|  
|     palette=None,  
|  
|     row_order=None,  
|  
|     col_order=None,  
|  
|     hue_order=None,  
|  
|     hue_kws=None,  
|  
|     dropna=False,  
|  
|     legend_out=True,  
|  
|     despine=True,  
|  
|     margin_titles=False,  
|  
|     xlim=None,  
|  
|     ylim=None,  
|  
|     subplot_kws=None,  
|  
|     gridspec_kws=None  
| )
```

```
| Initialize the matplotlib figure and FacetGrid object.
```

```
|
```

```
| This class maps a dataset onto multiple axes arrayed in a grid of rows  
| and columns that correspond to *levels* of variables in the dataset.
```

```
| The plots it produces are often called "lattice", "trellis", or  
| "small-multiple" graphics.
```

```
|
```

It can also represent levels of a third variable with the `hue` parameter, which plots different subsets of data in different colors. This uses color to resolve elements on a third dimension, but only draws subsets on top of each other and will not tailor the `hue` parameter for the specific visualization the way that axes-level functions that accept `hue` will.

The basic workflow is to initialize the `FacetGrid` object with the dataset and the variables that are used to structure the grid. Then one or more plotting functions can be applied to each subset by calling `FacetGrid.map` or `FacetGrid.map_dataframe`. Finally, the plot can be tweaked with other methods to do things like change the axis labels, use different ticks, or add a legend. See the detailed code examples below for more information.

.. warning::

When using seaborn functions that infer semantic mappings from a dataset, care must be taken to synchronize those mappings across facets (e.g., by defining the `hue` mapping with a palette dict or setting the data type of the variables to `category`). In most cases, it will be better to use a figure-level function (e.g. `relplot` or `catplot`) than to use `FacetGrid` directly.

See the [tutorial <grid_tutorial>](#) for more information.

Parameters

```

| -----
|
| data : DataFrame
|
|     Tidy ("long-form") dataframe where each column is a variable and each
|
|     row is an observation.
|
| row, col, hue : strings
|
|     Variables that define subsets of the data, which will be drawn on
|
|     separate facets in the grid. See the ``{var}_order`` parameters to
|
|     control the order of levels of this variable.
|
| col_wrap : int
|
|     "Wrap" the column variable at this width, so that the column facets
|
|     span multiple rows. Incompatible with a ``row`` facet.
|
| share{x,y} : bool, 'col', or 'row' optional
|
|     If true, the facets will share y axes across columns and/or x axes
|
|     across rows.
|
| height : scalar
|
|     Height (in inches) of each facet. See also: ``aspect``.
|
| aspect : scalar
|
|     Aspect ratio of each facet, so that ``aspect * height`` gives the width
|
|     of each facet in inches.
|
| palette : palette name, list, or dict
|
|     Colors to use for the different levels of the ``hue`` variable. Should
|
|     be something that can be interpreted by :func:`color_palette`, or a
|
|     dictionary mapping hue levels to matplotlib colors.
|
| {row,col,hue}_order : lists
|
|     Order for the levels of the faceting variables. By default, this
|
|     will be the order that the levels appear in ``data`` or, if the
|
|     variables are pandas categoricals, the category order.

```


| hue_kws : dictionary of param -> list of values mapping

| Other keyword arguments to insert into the plotting call to let

| other plot attributes vary across levels of the hue variable (e.g.

| the markers in a scatterplot).

| legend_out : bool

| If ``True``, the figure size will be extended, and the legend will be

| drawn outside the plot on the center right.

| despine : boolean

| Remove the top and right spines from the plots.

| margin_titles : bool

| If ``True``, the titles for the row variable are drawn to the right of

| the last column. This option is experimental and may not work in all

| cases.

| {x, y}lim: tuples

| Limits for each of the axes on each facet (only relevant when

| share{x, y} is True).

| subplot_kws : dict

| Dictionary of keyword arguments passed to matplotlib subplot(s)

| methods.

| gridspec_kws : dict

| Dictionary of keyword arguments passed to

| :class:`matplotlib.gridspec.GridSpec`

| (via :meth:`matplotlib.figure.Figure.subplots`).

| Ignored if ``col\_wrap`` is not ``None``.

|

| See Also

| -----

```

|     PairGrid : Subplot grid for plotting pairwise relationships
|
|     relplot : Combine a relational plot and a :class:`FacetGrid`
|
|     displot : Combine a distribution plot and a :class:`FacetGrid`
|
|     catplot : Combine a categorical plot and a :class:`FacetGrid`
|
|     lmplot : Combine a regression plot and a :class:`FacetGrid`
|
|
|     Examples
|
|     -----
|
|     .. note::
|
|         These examples use seaborn functions to demonstrate some of the
|         advanced features of the class, but in most cases you will want
|         to use figure-level functions (e.g. :func:`displot`, :func:`relplot`)
|         to make the plots shown here.
|
|     .. include:: ../docstrings/FacetGrid.rst
|
|     despine(self, **kwargs)
|
|         Remove axis spines from the facets.
|
|     facet_axis(self, row_i, col_j, modify_state=True)
|
|         Make the axis identified by these indices active and return it.
|
|     facet_data(self)
|
|         Generator for name indices and data subsets for each facet.
|

```

```

|     Yields
|
|     -----
|
|     (i, j, k), data_ijk : tuple of ints, DataFrame
|
|         The ints provide an index into the {row, col, hue}_names attribute,
|
|         and the dataframe contains a subset of the full data corresponding
|
|         to each facet. The generator yields subsets that correspond with
|
|         the self.axes.flat iterator, or self.axes[i, j] when `col_wrap`
|
|         is None.
|
|
|     map(self, func, *args, **kwargs)
|
|         Apply a plotting function to each facet's subset of the data.
|
|
|     Parameters
|
|     -----
|
|     func : callable
|
|         A plotting function that takes data and keyword arguments. It
|
|         must plot to the currently active matplotlib Axes and take a
|
|         `color` keyword argument. If faceting on the `hue` dimension,
|
|         it must also take a `label` keyword argument.
|
|     args : strings
|
|         Column names in self.data that identify variables with data to
|
|         plot. The data for each variable is passed to `func` in the
|
|         order the variables are specified in the call.
|
|     kwargs : keyword arguments
|
|         All keyword arguments are passed to the plotting function.
|
|
|     Returns

```

```

|         -----
|
|         self : object
|
|         Returns self.
|
|
|
| map_dataframe(self, func, *args, **kwargs)
|
|     Like ``.map`` but passes args as strings and inserts data in kwargs.
|
|
|     This method is suitable for plotting with functions that accept a
|
|     long-form DataFrame as a `data` keyword argument and access the
|
|     data in that DataFrame using string variable names.
|
|
|     Parameters
|
|     -----
|
|     func : callable
|
|         A plotting function that takes data and keyword arguments. Unlike
|
|         the `map` method, a function used here must "understand" Pandas
|
|         objects. It also must plot to the currently active matplotlib Axes
|
|         and take a `color` keyword argument. If faceting on the `hue`
|
|         dimension, it must also take a `label` keyword argument.
|
|     args : strings
|
|         Column names in self.data that identify variables with data to
|
|         plot. The data for each variable is passed to `func` in the
|
|         order the variables are specified in the call.
|
|     kwargs : keyword arguments
|
|         All keyword arguments are passed to the plotting function.
|
|
|
|     Returns

```

```

|         -----
|
|         self : object
|
|         Returns self.
|
|
|
|
|         refline(self, *, x=None, y=None, color='.5', linestyle='--', **line_kws)
|
|         Add a reference line(s) to each facet.
|
|
|         Parameters
|
|         -----
|
|         x, y : numeric
|
|         Value(s) to draw the line(s) at.
|
|         color : :mod:`matplotlib color <matplotlib.colors>`
|
|         Specifies the color of the reference line(s). Pass ``color=None`` to
|
|         use ``hue`` mapping.
|
|         linestyle : str
|
|         Specifies the style of the reference line(s).
|
|         line_kws : key, value mappings
|
|         Other keyword arguments are passed to :meth:`matplotlib.axes.Axes.axvline`
|
|         when ``x`` is not None and :meth:`matplotlib.axes.Axes.axhline` when ``y``
|
|         is not None.
|
|
|
|         Returns
|
|         -----
|
|         :class:`FacetGrid` instance
|
|         Returns ``self`` for easy method chaining.
|
|
|
|         set_axis_labels(self, x_var=None, y_var=None, clear_inner=True, **kwargs)

```

```

|     Set axis labels on the left column and bottom row of the grid.
|
|
| set_titles(self, template=None, row_template=None, col_template=None, **kwargs)
|
|     Draw titles either above each facet or on the grid margins.
|
|
|     Parameters
|
|     -----
|
|     template : string
|
|         Template for all titles with the formatting keys {col_var} and
|
|         {col_name} (if using a `col` faceting variable) and/or {row_var}
|
|         and {row_name} (if using a `row` faceting variable).
|
|     row_template:
|
|         Template for the row variable when titles are drawn on the grid
|
|         margins. Must have {row_var} and {row_name} formatting keys.
|
|     col_template:
|
|         Template for the column variable when titles are drawn on the grid
|
|         margins. Must have {col_var} and {col_name} formatting keys.
|
|
|
|     Returns
|
|     -----
|
|     self: object
|
|         Returns self.
|
|
|
| set_xlabel(self, label=None, clear_inner=True, **kwargs)
|
|     Label the x axis on the bottom row of the grid.
|
|
|
| set_xticklabels(self, labels=None, step=None, **kwargs)

```

```

|     Set x axis tick labels of the grid.
|
|
|     set_ylabels(self, label=None, clear_inner=True, **kwargs)
|
|     Label the y axis on the left column of the grid.
|
|
|     set_yticklabels(self, labels=None, **kwargs)
|
|     Set y axis tick labels on the left column of the grid.
|
|
|     -----
|
|     Readonly properties defined here:
|
|
|     ax
|
|     The :class:`matplotlib.axes.Axes` when no faceting variables are assigned.
|
|
|     axes
|
|     An array of the :class:`matplotlib.axes.Axes` objects in the grid.
|
|
|     axes_dict
|
|     A mapping of facet names to corresponding :class:`matplotlib.axes.Axes`.
|
|
|     If only one of ``row`` or ``col`` is assigned, each key is a string
|
|     representing a level of that variable. If both facet dimensions are
|
|     assigned, each key is a ``({row_level}, {col_level})`` tuple.
|
|
|     -----
|
|     Methods inherited from Grid:
|
|

```

```

|  add_legend(
|
|      self,
|
|      legend_data=None,
|
|      title=None,
|
|      label_order=None,
|
|      adjust_subtitles=False,
|
|      **kwargs
|  )

```

Draw a legend, maybe placing it outside axes and resizing the figure.

Parameters

legend_data : dict

Dictionary mapping label names (or two-element tuples where the second element is a label name) to matplotlib artist handles. The default reads from ``self.\_legend\_data``.

title : string

Title for the legend. The default reads from ``self.\_hue\_var``.

label_order : list of labels

The order that the legend entries should appear in. The default reads from ``self.hue\_names``.

adjust_subtitles : bool

If True, modify entries with invisible artists to left-align the labels and set the font size to that of a title.

kwargs : key, value pairings

Other keyword arguments are passed to the underlying legend methods on the Figure or Axes object.


```

|
|
| Returns
|
| -----
|
| self : Grid instance
|
| Returns self for easy chaining.
|
|
| tick_params(self, axis='both', **kwargs)
|
| Modify the ticks, tick labels, and gridlines.
|
|
| Parameters
|
| -----
|
| axis : {'x', 'y', 'both'}
|
| The axis on which to apply the formatting.
|
| kwargs : keyword arguments
|
| Additional keyword arguments to pass to
|
| :meth:`matplotlib.axes.Axes.tick_params`.
|
|
| Returns
|
| -----
|
| self : Grid instance
|
| Returns self for easy chaining.
|
|
| tight_layout(self, *args, **kwargs)
|
| Call fig.tight_layout within rect that exclude the legend.
|
|
| -----
|
| Readonly properties inherited from Grid:

```

```

|
|  legend
|
|      The :class:`matplotlib.legend.Legend` object, if present.
|
|
|  -----
|
|  Methods inherited from _BaseGrid:
|
|
|  apply(self, func, *args, **kwargs)
|
|      Pass the grid to a user-supplied function and return self.
|
|
|      The `func` must accept an object of this type for its first
|      positional argument. Additional arguments are passed through.
|
|      The return value of `func` is ignored; this method returns self.
|
|      See the `pipe` method if you want the return value.
|
|
|      Added in v0.12.0.
|
|
|  pipe(self, func, *args, **kwargs)
|
|      Pass the grid to a user-supplied function and return its value.
|
|
|      The `func` must accept an object of this type for its first
|      positional argument. Additional arguments are passed through.
|
|      The return value of `func` becomes the return value of this method.
|
|      See the `apply` method if you want to return self instead.
|
|
|      Added in v0.12.0.
|

```

```
| savefig(self, *args, **kwargs)

|     Save an image of the plot.

|

|     This wraps :meth:`matplotlib.figure.Figure.savefig`, using bbox_inches="tight"
|     by default. Parameters are passed through to the matplotlib function.

|

| set(self, **kwargs)

|     Set attributes on each subplot Axes.

|

| -----

| Readonly properties inherited from _BaseGrid:

|

| fig

|     DEPRECATED: prefer the `figure` property.

|

| figure

|     Access the :class:`matplotlib.figure.Figure` object underlying the grid.

|

| -----

| Data descriptors inherited from _BaseGrid:

|

| __dict__

|     dictionary for instance variables

|

| __weakref__

|     list of weak references to the object
```

```
class JointGrid(_BaseGrid)
```

```
| JointGrid(  
  
|     data=None,  
  
|     *,  
  
|     x=None,  
  
|     y=None,  
  
|     hue=None,  
  
|     height=6,  
  
|     ratio=5,  
  
|     space=0.2,  
  
|     palette=None,  
  
|     hue_order=None,  
  
|     hue_norm=None,  
  
|     dropna=False,  
  
|     xlim=None,  
  
|     ylim=None,  
  
|     marginal_ticks=False  
  
| )
```

```
|
```

```
| Grid for drawing a bivariate plot with marginal univariate plots.
```

```
|
```

```
| Many plots can be drawn by using the figure-level interface :func:`jointplot`.
```

```
| Use this class directly when you need more flexibility.
```

```
|
```

```
| Method resolution order:
```

```
|     JointGrid
```

```
|     _BaseGrid
```

```
|      builtins.object
|
|  Methods defined here:
|
|  __init__(
|      self,
|      data=None,
|      *,
|      x=None,
|      y=None,
|      hue=None,
|      height=6,
|      ratio=5,
|      space=0.2,
|      palette=None,
|      hue_order=None,
|      hue_norm=None,
|      dropna=False,
|      xlim=None,
|      ylim=None,
|      marginal_ticks=False
|  )
|
|  Set up the grid of subplots and store data internally for easy plotting.
|
|  Parameters
|  -----
|
|  data : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence
```

| Input data structure. Either a long-form collection of vectors that can be

| assigned to named variables or a wide-form dataset that will be internally

| reshaped.

| x, y : vectors or keys in ``data``

| Variables that specify positions on the x and y axes.

| height : number

| Size of each side of the figure in inches (it will be square).

| ratio : number

| Ratio of joint axes height to marginal axes height.

| space : number

| Space between the joint and marginal axes

| dropna : bool

| If True, remove missing observations before plotting.

| {x, y}lim : pairs of numbers

| Set axis limits to these values before plotting.

| marginal_ticks : bool

| If False, suppress ticks on the count/density axis of the marginal plots.

| hue : vector or key in ``data``

| Semantic variable that is mapped to determine the color of plot elements.

| Note: unlike in :class:`FacetGrid` or :class:`PairGrid`, the axes-level

| functions must support ``hue`` to use it in :class:`JointGrid`.

| palette : string, list, dict, or :class:`matplotlib.colors.Colormap`

| Method for choosing the colors to use when mapping the ``hue`` semantic.

| String values are passed to :func:`color\_palette`. List or dict values

| imply categorical mapping, while a colormap object implies numeric mapping.

| hue_order : vector of strings

| Specify the order of processing and plotting for categorical levels of the

| ``hue`` semantic.

| hue_norm : tuple or :class:`matplotlib.colors.Normalize`

| Either a pair of values that set the normalization range in data units

| or an object that will map from data units into a [0, 1] interval. Usage

| implies numeric mapping.

|

| See Also

| -----

| jointplot : Draw a bivariate plot with univariate marginal distributions.

| PairGrid : Set up a figure with joint and marginal views on multiple variables.

| jointplot : Draw multiple bivariate plots with univariate marginal distributions.

|

| Examples

| -----

|

| .. include:: ../docstrings/JointGrid.rst

|

| plot(self, joint_func, marginal_func, **kwargs)

| Draw the plot by passing functions for joint and marginal axes.

|

| This method passes the ``kwargs`` dictionary to both functions. If you

| need more control, call :meth:`JointGrid.plot\_joint` and

| :meth:`JointGrid.plot\_marginals` directly with specific parameters.

|

| Parameters

| -----

| joint_func, marginal_func : callables

| Functions to draw the bivariate and univariate plots. See methods
| referenced above for information about the required characteristics
| of these functions.

| **kwargs**

| Additional keyword arguments are passed to both functions.

| **Returns**

| -----

| :class:`JointGrid` instance

| Returns ``self`` for easy method chaining.

| **plot_joint(self, func, **kwargs)**

| Draw a bivariate plot on the joint axes of the grid.

| **Parameters**

| -----

| **func** : plotting callable

| If a seaborn function, it should accept ``x`` and ``y``. Otherwise,
| it must accept ``x`` and ``y`` vectors of data as the first two
| positional arguments, and it must plot on the "current" axes.

| If ``hue`` was defined in the class constructor, the function must
| accept ``hue`` as a parameter.

| **kwargs**

| Keyword argument are passed to the plotting function.

| **Returns**

| -----


```

|         :class:`JointGrid` instance
|
|         Returns ``self`` for easy method chaining.
|
|
| plot_marginals(self, func, **kwargs)
|
|     Draw univariate plots on each marginal axes.
|
|
|     Parameters
|
|     -----
|
|     func : plotting callable
|
|         If a seaborn function, it should accept ``x`` and ``y`` and plot
|
|         when only one of them is defined. Otherwise, it must accept a vector
|
|         of data as the first positional argument and determine its orientation
|
|         using the ``vertical`` parameter, and it must plot on the "current" axes.
|
|         If ``hue`` was defined in the class constructor, it must accept ``hue``
|
|         as a parameter.
|
|     kwargs
|
|         Keyword argument are passed to the plotting function.
|
|
|     Returns
|
|     -----
|
|     :class:`JointGrid` instance
|
|         Returns ``self`` for easy method chaining.
|
|
|
|     refline(
|
|         self,
|
|         *,
|
|         x=None,

```

```

|     y=None,
|
|     joint=True,
|
|     marginal=True,
|
|     color='.5',
|
|     linestyle='--',
|
|     **line_kws
| )
|
| Add a reference line(s) to joint and/or marginal axes.
|
|
| Parameters
|
| -----
|
| x, y : numeric
|
|     Value(s) to draw the line(s) at.
|
| joint, marginal : bools
|
|     Whether to add the reference line(s) to the joint/marginal axes.
|
| color : :mod:`matplotlib color <matplotlib.colors>`
|
|     Specifies the color of the reference line(s).
|
| linestyle : str
|
|     Specifies the style of the reference line(s).
|
| line_kws : key, value mappings
|
|     Other keyword arguments are passed to :meth:`matplotlib.axes.Axes.axvline`
|     when ``x`` is not None and :meth:`matplotlib.axes.Axes.axhline` when ``y``
|     is not None.
|
|
| Returns
|
| -----
|
| :class:`JointGrid` instance

```

```

|         Returns ``self`` for easy method chaining.
|
|
| set_axis_labels(self, xlabel='', ylabel='', **kwargs)
|
|     Set axis labels on the bivariate axes.
|
|
|     Parameters
|
|     -----
|
|     xlabel, ylabel : strings
|
|         Label names for the x and y variables.
|
|     kwargs : key, value mappings
|
|         Other keyword arguments are passed to the following functions:
|
|
|         - :meth:`matplotlib.axes.Axes.set_xlabel`
|
|         - :meth:`matplotlib.axes.Axes.set_ylabel`
|
|
|
|     Returns
|
|     -----
|
|     :class:`JointGrid` instance
|
|         Returns ``self`` for easy method chaining.
|
|
| -----
|
| Methods inherited from _BaseGrid:
|
|
| apply(self, func, *args, **kwargs)
|
|     Pass the grid to a user-supplied function and return self.
|
|
|
|     The `func` must accept an object of this type for its first

```

| positional argument. Additional arguments are passed through.

| The return value of `func` is ignored; this method returns self.

| See the `pipe` method if you want the return value.

|

| Added in v0.12.0.

|

| pipe(self, func, *args, **kwargs)

| Pass the grid to a user-supplied function and return its value.

|

| The `func` must accept an object of this type for its first

| positional argument. Additional arguments are passed through.

| The return value of `func` becomes the return value of this method.

| See the `apply` method if you want to return self instead.

|

| Added in v0.12.0.

|

| savefig(self, *args, **kwargs)

| Save an image of the plot.

|

| This wraps :meth:`matplotlib.figure.Figure.savefig`, using `bbox_inches="tight"`

| by default. Parameters are passed through to the matplotlib function.

|

| set(self, **kwargs)

| Set attributes on each subplot Axes.

|

| -----

| Readonly properties inherited from `_BaseGrid`:

```

|
|  fig
|
|      DEPRECATED: prefer the `figure` property.
|
|
|  figure
|
|      Access the :class:`matplotlib.figure.Figure` object underlying the grid.
|
|
|  -----
|
|  Data descriptors inherited from _BaseGrid:
|
|
|  __dict__
|
|      dictionary for instance variables
|
|
|  __weakref__
|
|      list of weak references to the object

```

```

class PairGrid(Grid)

```

```

|  PairGrid(
|
|      data,
|
|      *,
|
|      hue=None,
|
|      vars=None,
|
|      x_vars=None,
|
|      y_vars=None,
|
|      hue_order=None,
|
|      palette=None,
|
|      hue_kws=None,

```

```
|     corner=False,  
|  
|     diag_sharey=True,  
|  
|     height=2.5,  
|  
|     aspect=1,  
|  
|     layout_pad=0.5,  
|  
|     despine=True,  
|  
|     dropna=False  
| )
```

```
|  
| Subplot grid for plotting pairwise relationships in a dataset.  
|
```

```
| This object maps each variable in a dataset onto a column and row in a  
| grid of multiple axes. Different axes-level plotting functions can be  
| used to draw bivariate plots in the upper and lower triangles, and the  
| marginal distribution of each variable can be shown on the diagonal.  
|
```

```
| Several different common plots can be generated in a single line using  
| :func:`pairplot`. Use :class:`PairGrid` when you need more flexibility.  
|
```

```
| See the :ref:`tutorial <grid_tutorial>` for more information.  
|
```

```
| Method resolution order:
```

```
|     PairGrid  
|  
|     Grid  
|  
|     _BaseGrid  
|  
|     builtins.object  
|
```

| Methods defined here:

|

| `__init__`(

| `self`,

| `data`,

| `*`,

| `hue=None`,

| `vars=None`,

| `x_vars=None`,

| `y_vars=None`,

| `hue_order=None`,

| `palette=None`,

| `hue_kws=None`,

| `corner=False`,

| `diag_sharey=True`,

| `height=2.5`,

| `aspect=1`,

| `layout_pad=0.5`,

| `despine=True`,

| `dropna=False`

|)

| Initialize the plot figure and PairGrid object.

|

| Parameters

| -----

| `data` : DataFrame

| Tidy (long-form) dataframe where each column is a variable and

| each row is an observation.

| hue : string (variable name)

| Variable in ``data`` to map plot aspects to different colors. This

| variable will be excluded from the default x and y variables.

| vars : list of variable names

| Variables within ``data`` to use, otherwise use every column with

| a numeric datatype.

| {x, y}_vars : lists of variable names

| Variables within ``data`` to use separately for the rows and

| columns of the figure; i.e. to make a non-square plot.

| hue_order : list of strings

| Order for the levels of the hue variable in the palette

| palette : dict or seaborn color palette

| Set of colors for mapping the ``hue`` variable. If a dict, keys

| should be values in the ``hue`` variable.

| hue_kws : dictionary of param -> list of values mapping

| Other keyword arguments to insert into the plotting call to let

| other plot attributes vary across levels of the hue variable (e.g.

| the markers in a scatterplot).

| corner : bool

| If True, don't add axes to the upper (off-diagonal) triangle of the

| grid, making this a "corner" plot.

| height : scalar

| Height (in inches) of each facet.

| aspect : scalar

| Aspect * height gives the width (in inches) of each facet.

| layout_pad : scalar


```

|         Padding between axes; passed to ``fig.tight_layout``.
|
|     despine : boolean
|
|         Remove the top and right spines from the plots.
|
|     dropna : boolean
|
|         Drop missing values from the data before plotting.
|
|
|
|     See Also
|
|     -----
|
|     pairplot : Easily drawing common uses of :class:`PairGrid`.
|
|     FacetGrid : Subplot grid for plotting conditional relationships.
|
|
|
|     Examples
|
|     -----
|
|
|     .. include:: ../docstrings/PairGrid.rst
|
|
|
| map(self, func, **kwargs)
|
|     Plot with the same function in every subplot.
|
|
|
|     Parameters
|
|     -----
|
|     func : callable plotting function
|
|         Must take x, y arrays as positional arguments and draw onto the
|
|         "currently active" matplotlib Axes. Also needs to accept kwargs
|
|         called ``color`` and ``label``.
|
|
|
| map_diag(self, func, **kwargs)

```

```

|     Plot with a univariate function on each diagonal subplot.
|
|
|     Parameters
|
|     -----
|
|     func : callable plotting function
|
|           Must take an x array as a positional argument and draw onto the
|
|           "currently active" matplotlib Axes. Also needs to accept kwargs
|
|           called ``color`` and ``label``.
|
|
| map_lower(self, func, **kwargs)
|
|     Plot with a bivariate function on the lower diagonal subplots.
|
|
|     Parameters
|
|     -----
|
|     func : callable plotting function
|
|           Must take x, y arrays as positional arguments and draw onto the
|
|           "currently active" matplotlib Axes. Also needs to accept kwargs
|
|           called ``color`` and ``label``.
|
|
| map_offdiag(self, func, **kwargs)
|
|     Plot with a bivariate function on the off-diagonal subplots.
|
|
|     Parameters
|
|     -----
|
|     func : callable plotting function
|
|           Must take x, y arrays as positional arguments and draw onto the
|
|           "currently active" matplotlib Axes. Also needs to accept kwargs

```

```

|         called ``color`` and ``label``.
|
|
| map_upper(self, func, **kwargs)
|
|     Plot with a bivariate function on the upper diagonal subplots.
|
|
|     Parameters
|
|     -----
|
|     func : callable plotting function
|
|         Must take x, y arrays as positional arguments and draw onto the
|
|         "currently active" matplotlib Axes. Also needs to accept kwargs
|
|         called ``color`` and ``label``.
|
|
|     -----
|
| Methods inherited from Grid:
|
|
| add_legend(
|
|     self,
|
|     legend_data=None,
|
|     title=None,
|
|     label_order=None,
|
|     adjust_subtitles=False,
|
|     **kwargs
|
| )
|
|     Draw a legend, maybe placing it outside axes and resizing the figure.
|
|
|     Parameters
|
|     -----

```

```

|     legend_data : dict
|
|         Dictionary mapping label names (or two-element tuples where the
|         second element is a label name) to matplotlib artist handles. The
|         default reads from ``self._legend_data``.
|
|     title : string
|
|         Title for the legend. The default reads from ``self._hue_var``.
|
|     label_order : list of labels
|
|         The order that the legend entries should appear in. The default
|         reads from ``self.hue_names``.
|
|     adjust_subtitles : bool
|
|         If True, modify entries with invisible artists to left-align
|         the labels and set the font size to that of a title.
|
|     kwargs : key, value pairings
|
|         Other keyword arguments are passed to the underlying legend methods
|         on the Figure or Axes object.
|
|
|     Returns
|
|     -----
|
|     self : Grid instance
|
|         Returns self for easy chaining.
|
|
|     tick_params(self, axis='both', **kwargs)
|
|         Modify the ticks, tick labels, and gridlines.
|
|
|     Parameters
|
|     -----
|
|     axis : {'x', 'y', 'both'}

```

```

|         The axis on which to apply the formatting.

|     kwargs : keyword arguments

|         Additional keyword arguments to pass to

|         :meth:`matplotlib.axes.Axes.tick_params`.

|

|     Returns

|     -----

|     self : Grid instance

|         Returns self for easy chaining.

|

|     tight_layout(self, *args, **kwargs)

|         Call fig.tight_layout within rect that exclude the legend.

|

|     -----

|     Readonly properties inherited from Grid:

|

|     legend

|         The :class:`matplotlib.legend.Legend` object, if present.

|

|     -----

|     Methods inherited from _BaseGrid:

|

|     apply(self, func, *args, **kwargs)

|         Pass the grid to a user-supplied function and return self.

|

|         The `func` must accept an object of this type for its first

|         positional argument. Additional arguments are passed through.

```

| The return value of ``func`` is ignored; this method returns `self`.

| See the ``pipe`` method if you want the return value.

|

| Added in v0.12.0.

|

| `pipe(self, func, *args, **kwargs)`

| Pass the grid to a user-supplied function and return its value.

|

| The ``func`` must accept an object of this type for its first
| positional argument. Additional arguments are passed through.

| The return value of ``func`` becomes the return value of this method.

| See the ``apply`` method if you want to return `self` instead.

|

| Added in v0.12.0.

|

| `savefig(self, *args, **kwargs)`

| Save an image of the plot.

|

| This wraps `:meth:`matplotlib.figure.Figure.savefig``, using `bbox_inches="tight"`
| by default. Parameters are passed through to the matplotlib function.

|

| `set(self, **kwargs)`

| Set attributes on each subplot Axes.

|

| -----

| Readonly properties inherited from `_BaseGrid`:

|

```
| fig

|     DEPRECATED: prefer the `figure` property.

|

| figure

|     Access the :class:`matplotlib.figure.Figure` object underlying the grid.

|

| -----

| Data descriptors inherited from _BaseGrid:

|

| __dict__

|     dictionary for instance variables

|

| __weakref__

|     list of weak references to the object
```

FUNCTIONS

```
jointplot(

    data=None,

    *,

    x=None,

    y=None,

    hue=None,

    kind='scatter',

    height=6,

    ratio=5,

    space=0.2,

    dropna=False,
```

```

xlim=None,

ylim=None,

color=None,

palette=None,

hue_order=None,

hue_norm=None,

marginal_ticks=False,

joint_kws=None,

marginal_kws=None,

**kwargs
)

```

Draw a plot of two variables with bivariate and univariate graphs.

This function provides a convenient interface to the `:class:`JointGrid`` class, with several canned plot kinds. This is intended to be a fairly lightweight wrapper; if you need more flexibility, you should use `:class:`JointGrid`` directly.

Parameters

data : `:class:`pandas.DataFrame``, `:class:`numpy.ndarray``, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

x, y : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

hue : vector or key in ``data``

Semantic variable that is mapped to determine the color of plot elements.

kind : { "scatter" | "kde" | "hist" | "hex" | "reg" | "resid" }

Kind of plot to draw. See the examples for references to the underlying functions.

height : numeric

Size of the figure (it will be square).

ratio : numeric

Ratio of joint axes height to marginal axes height.

space : numeric

Space between the joint and marginal axes

dropna : bool

If True, remove observations that are missing from ``x`` and ``y``.

{x, y}lim : pairs of numbers

Axis limits to set before plotting.

color : :mod:`matplotlib color <matplotlib.colors>`

Single color specification for when hue mapping is not used. Otherwise, the plot will try to hook into the matplotlib property cycle.

palette : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to :func:`color\_palette`. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

hue_order : vector of strings

Specify the order of processing and plotting for categorical levels of the ``hue`` semantic.

hue_norm : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units or an object that will map from data units into a [0, 1] interval. Usage implies numeric mapping.

marginal_ticks : bool

If False, suppress ticks on the count/density axis of the marginal plots.

{joint, marginal}_kws : dicts

Additional keyword arguments for the plot components.

kwargs

Additional keyword arguments are passed to the function used to

draw the plot on the joint Axes, superseding items in the

```joint_kws``` dictionary.

Returns

-----

:class: `JointGrid`

An object managing multiple subplots that correspond to joint and marginal axes

for plotting a bivariate relationship or distribution.

See Also

-----

JointGrid : Set up a figure with joint and marginal views on bivariate data.

PairGrid : Set up a figure with joint and marginal views on multiple variables.

jointplot : Draw multiple bivariate plots with univariate marginal distributions.

Examples

-----

```
.. include:: ../docstrings/jointplot.rst
```

pairplot(

```
data,

*,

hue=None,

hue_order=None,

palette=None,

vars=None,

x_vars=None,

y_vars=None,

kind='scatter',

diag_kind='auto',

markers=None,

height=2.5,

aspect=1,

corner=False,

dropna=False,

plot_kws=None,

diag_kws=None,

grid_kws=None,

size=None
)
```

Plot pairwise relationships in a dataset.

By default, this function will create a grid of Axes such that each numeric variable in ``data`` will be shared across the y-axes across a single row and the x-axes across a single column. The diagonal plots are treated differently: a univariate distribution plot is drawn to show the marginal distribution of the data in each column.

It is also possible to show a subset of variables or plot different variables on the rows and columns.

This is a high-level interface for `:class:`PairGrid`` that is intended to make it easy to draw a few common styles. You should use `:class:`PairGrid`` directly if you need more flexibility.

#### Parameters

-----

`data` : ``pandas.DataFrame``

Tidy (long-form) dataframe where each column is a variable and each row is an observation.

`hue` : name of variable in ```data```

Variable in ```data``` to map plot aspects to different colors.

`hue_order` : list of strings

Order for the levels of the hue variable in the palette

`palette` : dict or seaborn color palette

Set of colors for mapping the ```hue``` variable. If a dict, keys should be values in the ```hue``` variable.

`vars` : list of variable names

Variables within ```data``` to use, otherwise use every column with a numeric datatype.

`{x, y}_vars` : lists of variable names

Variables within ```data``` to use separately for the rows and columns of the figure; i.e. to make a non-square plot.

`kind` : {'scatter', 'kde', 'hist', 'reg'}

Kind of plot to make.

`diag_kind : {'auto', 'hist', 'kde', None}`

Kind of plot for the diagonal subplots. If 'auto', choose based on whether or not ``hue`` is used.

`markers : single matplotlib marker code or list`

Either the marker to use for all scatterplot points or a list of markers with a length the same as the number of levels in the hue variable so that differently colored points will also have different scatterplot markers.

`height : scalar`

Height (in inches) of each facet.

`aspect : scalar`

Aspect \* height gives the width (in inches) of each facet.

`corner : bool`

If True, don't add axes to the upper (off-diagonal) triangle of the grid, making this a "corner" plot.

`dropna : boolean`

Drop missing values from the data before plotting.

`{plot, diag, grid}_kws : dicts`

Dictionaries of keyword arguments. ``plot\_kws`` are passed to the bivariate plotting function, ``diag\_kws`` are passed to the univariate plotting function, and ``grid\_kws`` are passed to the :class:`PairGrid` constructor.

Returns

-----

`grid : :class:`PairGrid``

Returns the underlying `:class:`PairGrid`` instance for further tweaking.

See Also

-----

`PairGrid` : Subplot grid for more flexible plotting of pairwise relationships.

`JointGrid` : Grid for plotting joint and marginal distributions of two variables.

Examples

-----

```
.. include:: ../docstrings/pairplot.rst
```

## DATA

```
__all__ = ['FacetGrid', 'PairGrid', 'JointGrid', 'pairplot', 'jointplo...
```

## FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\axisgrid.py
```

Help on module `seaborn.categorical` in `seaborn`:

## NAME

```
seaborn.categorical
```

## FUNCTIONS

```
barplot(
```

```
 data=None,
```

```
* ,

x=None,

y=None,

hue=None,

order=None,

hue_order=None,

estimator='mean',

errorbar=('ci', 95),

n_boot=1000,

seed=None,

units=None,

weights=None,

orient=None,

color=None,

palette=None,

saturation=0.75,

fill=True,

hue_norm=None,

width=0.8,

dodge='auto',

gap=0,

log_scale=None,

native_scale=False,

formatter=None,

legend='auto',

capsize=0,

err_kws=None,
```

```

ci=<deprecated>,

errcolor=<deprecated>,

errwidth=<deprecated>,

ax=None,

**kwargs
)

```

Show point estimates and errors as rectangular bars.

A bar plot represents an aggregate or statistical estimate for a numeric variable with the height of each rectangle and indicates the uncertainty around that estimate using an error bar. Bar plots include 0 in the axis range, and they are a good choice when 0 is a meaningful value for the variable to take.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting `native_scale=True`.

Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If `x` and `y` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.



`x, y, hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order, hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`estimator` : string or callable that maps vector -> scalar

Statistical function to estimate within each categorical bin.

`errorbar` : string, (string, number) tuple, callable or None

Name of errorbar method (either "ci", "pi", "se", or "sd"), or a tuple with a method name and a level parameter, or a function that maps from a vector to a (min, max) interval, or None to hide errorbar. See the :doc:`errorbar tutorial </tutorial/errorBars>` for more information.

.. versionadded:: v0.12.0

`n_boot` : int

Number of bootstrap samples used to compute confidence intervals.

`seed` : int, ``numpy.random.Generator``, or ``numpy.random.RandomState``

Seed or random number generator for reproducible bootstrapping.

`units` : name of variable in ``data`` or vector data

Identifier of sampling units; used by the errorbar function to perform a multilevel bootstrap and account for repeated measures

`weights` : name of variable in ``data`` or vector data

Data values or column used to compute weighted statistics.

Note that the use of weights may limit other statistical options.

.. versionadded:: v0.13.1

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both ``x`` and ``y`` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added `'x'/'y'` as options, equivalent to `'v'/'h'`.

`color` : matplotlib color

Single color for the elements in the plot.

`palette` : palette name, list, or dict

Colors to use for the different levels of the ```hue``` variable. Should be something that can be interpreted by `:func:`color_palette``, or a dictionary mapping hue levels to matplotlib colors.

`saturation` : float

Proportion of the original saturation to draw fill colors in. Large patches often look better with desaturated colors, but set this to ``1`` if you want the colors to perfectly match the input values.

`fill` : bool

If True, use a solid patch. Otherwise, draw as line art.

.. versionadded:: v0.13.0

`hue_norm` : tuple or `:class:`matplotlib.colors.Normalize`` object

Normalization in data units for colormap applied to the ```hue``` variable when it is numeric. Not relevant if ```hue``` is categorical.

.. versionadded:: v0.12.0

`width` : float

Width allotted to each element on the orient axis. When `native_scale=True`, it is relative to the minimum distance between two values in the native scale.

`dodge` : "auto" or bool

When hue mapping is used, whether elements should be narrowed and shifted along the orient axis to eliminate overlap. If `"auto"`, set to `True` when the orient variable is crossed with the categorical variable or `False` otherwise.

.. versionchanged:: 0.13.0

Added `"auto"` mode as a new default.

`gap` : float

Shrink on the orient axis by this factor to add a gap between dodged elements.

.. versionadded:: 0.13.0

`log_scale` : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When `None` or `False`, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

`native_scale` : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

`formatter` : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

capsize : float

Width of the "caps" on error bars, relative to bar spacing.

err\_kws : dict

Parameters of `:class:`matplotlib.lines.Line2D``, for the error bar artists.

.. versionadded:: v0.13.0

ci : float

Level of the confidence interval to show, in `[0, 100]`.

.. deprecated:: v0.12.0

Use ``errorbar=("ci", ...)``.

errcolor : matplotlib color

Color used for the error bar lines.

.. deprecated:: 0.13.0

Use ``err_kws={'color': ...}``.

`errwidth` : float

Thickness of error bar lines (and caps), in points.

.. deprecated:: 0.13.0

Use ``err_kws={'linewidth': ...}``.

`ax` : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

`kwargs` : key, value mappings

Other parameters are passed through to `:class:`matplotlib.patches.Rectangle``.

## Returns

-----

`ax` : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

## See Also

-----

`countplot` : Show the counts of observations in each categorical bin.

`pointplot` : Show point estimates and confidence intervals using dots.

`catplot` : Combine a categorical plot with a `:class:`FacetGrid``.

## Notes

-----

For datasets where 0 is not a meaningful value, a `:func:`pointplot`` will

allow you to focus on differences between levels of one or more categorical

variables.

It is also important to keep in mind that a bar plot shows only the mean (or other aggregate) value, but it is often more informative to show the distribution of values at each level of the categorical variables. In those cases, approaches such as a `:func:`boxplot`` or `:func:`violinplot`` may be more appropriate.

Examples

-----

```
.. include:: ../docstrings/barplot.rst
```

```
boxenplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 order=None,

 hue_order=None,

 orient=None,

 color=None,

 palette=None,

 saturation=0.75,

 fill=True,

 dodge='auto',

 width=0.8,
```

```
gap=0,

linewidth=None,

linecolor=None,

width_method='exponential',

k_depth='tukey',

outlier_prop=0.007,

trust_alpha=0.05,

showfliers=True,

hue_norm=None,

log_scale=None,

native_scale=False,

formatter=None,

legend='auto',

scale=<deprecated>,

box_kws=None,

flier_kws=None,

line_kws=None,

ax=None,

**kwargs

)
```

Draw an enhanced box plot for larger datasets.

This style of plot was originally named a "letter value" plot because it shows a large number of quantiles that are defined as "letter values". It is similar to a box plot in plotting a nonparametric representation of a distribution in which all features correspond to actual observations. By plotting more quantiles, it provides more information about the shape of

the distribution, particularly in the tails.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting `native_scale=True`.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If `x` and `y` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x, y, hue` : names of variables in `data` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order, hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both `x` and `y` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0



Added 'x'/'y' as options, equivalent to 'v'/'h'.

color : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

saturation : float

Proportion of the original saturation to draw fill colors in. Large patches often look better with desaturated colors, but set this to `1` if you want the colors to perfectly match the input values.

fill : bool

If True, use a solid patch. Otherwise, draw as line art.

.. versionadded:: v0.13.0

dodge : "auto" or bool

When hue mapping is used, whether elements should be narrowed and shifted along the orient axis to eliminate overlap. If ``"auto"`, set to `True` when the orient variable is crossed with the categorical variable or `False` otherwise.

.. versionchanged:: 0.13.0

Added ``"auto"`` mode as a new default.

width : float

Width allotted to each element on the orient axis. When `native\_scale=True`, it is relative to the minimum distance between two values in the native scale.

gap : float

Shrink on the orient axis by this factor to add a gap between dodged elements.

.. versionadded:: 0.13.0

linewidth : float

Width of the lines that frame the plot elements.

linecolor : color

Color to use for line elements, when `fill` is True.

.. versionadded:: v0.13.0

width\_method : {"exponential", "linear", "area"}

Method to use for the width of the letter value boxes:

- `"exponential"`: Represent the corresponding percentile
- `"linear"`: Decrease by a constant amount for each box
- `"area"`: Represent the density of data points in that box

k\_depth : {"tukey", "proportion", "trustworthy", "full"} or int

The number of levels to compute and draw in each tail:

- `"tukey"`: Use  $\log_2(n) - 3$  levels, covering similar range as boxplot whiskers
- `"proportion"`: Leave approximately `outlier_prop` fliers
- `"trustworthy"`: Extend to level with confidence of at least `trust_alpha`
- `"full"`: Use  $\log_2(n) + 1$  levels and extend to most extreme points

outlier\_prop : float

Proportion of data expected to be outliers; used when `k_depth="proportion"`.

trust\_alpha : float

Confidence threshold for most extreme level; used when `k_depth="trustworthy"`.

showfliers : bool

If False, suppress the plotting of outliers.

hue\_norm : tuple or :class:`matplotlib.colors.Normalize` object

Normalization in data units for colormap applied to the `hue`

variable when it is numeric. Not relevant if `hue` is categorical.

.. versionadded:: v0.12.0

log\_scale : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric

axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When `None` or `False`, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

native\_scale : bool

When True, numeric or datetime values on the categorical axis will maintain

their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

formatter : callable

Function for converting categorical data into strings. Affects both grouping

and tick labels.

.. versionadded:: v0.13.0

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric `hue` and `size`

variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto",

choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

`box_kws`: dict

Keyword arguments for the box artists; passed to

`:class:`matplotlib.patches.Rectangle``.

.. versionadded:: v0.12.0

`line_kws`: dict

Keyword arguments for the line denoting the median; passed to

`:meth:`matplotlib.axes.Axes.plot``.

.. versionadded:: v0.12.0

`flier_kws`: dict

Keyword arguments for the scatter denoting the outlier observations;

passed to `:meth:`matplotlib.axes.Axes.scatter``.

.. versionadded:: v0.12.0

`ax` : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

`kwargs` : key, value mappings

Other keyword arguments are passed to `:class:`matplotlib.patches.Rectangle``,

superceded by those in ``box_kws``.

Returns

-----

ax : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

See Also

-----

violinplot : A combination of boxplot and kernel density estimation.

boxplot : A traditional box-and-whisker plot with a similar API.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

Notes

-----

For a more extensive explanation, you can read the paper that introduced the plot:

<https://vita.had.co.nz/papers/letter-value-plot.html>

Examples

-----

```
.. include:: ../docstrings/boxenplot.rst
```

```
boxplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 order=None,

 hue_order=None,
```

```
orient=None,

color=None,

palette=None,

saturation=0.75,

fill=True,

dodge='auto',

width=0.8,

gap=0,

whis=1.5,

linecolor='auto',

linewidth=None,

fliersize=None,

hue_norm=None,

native_scale=False,

log_scale=None,

formatter=None,

legend='auto',

ax=None,

**kwargs
)
```

Draw a box plot to show distributions with respect to categories.

A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable. The box shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution, except for points that are determined to be "outliers" using a method

that is a function of the inter-quartile range.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting `native_scale=True`.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If `x` and `y` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x, y, hue` : names of variables in `data` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order, hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both `x` and `y` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added 'x'/'y' as options, equivalent to 'v'/'h'.

color : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

saturation : float

Proportion of the original saturation to draw fill colors in. Large patches often look better with desaturated colors, but set this to `1` if you want the colors to perfectly match the input values.

fill : bool

If True, use a solid patch. Otherwise, draw as line art.

.. versionadded:: v0.13.0

dodge : "auto" or bool

When hue mapping is used, whether elements should be narrowed and shifted along the orient axis to eliminate overlap. If ``"auto"`, set to `True` when the orient variable is crossed with the categorical variable or `False` otherwise.

.. versionchanged:: 0.13.0

Added ``"auto"`` mode as a new default.

width : float

Width allotted to each element on the orient axis. When `native\_scale=True`, it is relative to the minimum distance between two values in the native scale.

gap : float



Shrink on the orient axis by this factor to add a gap between dodged elements.

.. versionadded:: 0.13.0

`whis` : float or pair of floats

Parameter that controls whisker length. If scalar, whiskers are drawn to the farthest datapoint within  $*whis * IQR*$  from the nearest hinge. If a tuple, it is interpreted as percentiles that whiskers represent.

`linecolor` : color

Color to use for line elements, when `fill` is True.

.. versionadded:: v0.13.0

`linewidth` : float

Width of the lines that frame the plot elements.

`fliersize` : float

Size of the markers used to indicate outlier observations.

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize` object

Normalization in data units for colormap applied to the `hue` variable when it is numeric. Not relevant if `hue` is categorical.

.. versionadded:: v0.12.0

`log_scale` : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently. Numeric values are interpreted as the desired base (default 10). When `None` or `False`, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

`native_scale : bool`

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

`formatter : callable`

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

`legend : "auto", "brief", "full", or False`

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

`ax : matplotlib Axes`

Axes object to draw the plot onto, otherwise uses the current Axes.

`kwargs : key, value mappings`

Other keyword arguments are passed through to

`:meth:`matplotlib.axes.Axes.boxplot``.

Returns

-----

`ax : matplotlib Axes`

Returns the Axes object with the plot drawn onto it.

See Also

-----

violinplot : A combination of boxplot and kernel density estimation.

stripplot : A scatterplot where one variable is categorical. Can be used  
in conjunction with other plots to show each observation.

swarmplot : A categorical scatterplot where the points do not overlap. Can  
be used with other plots to show each observation.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

Examples

-----

```
.. include:: ../docstrings/boxplot.rst
```

```
catplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 row=None,
 col=None,
 kind='strip',
 estimator='mean',
 errorbar=('ci', 95),
 n_boot=1000,
```

```
seed=None,

units=None,

weights=None,

order=None,

hue_order=None,

row_order=None,

col_order=None,

col_wrap=None,

height=5,

aspect=1,

log_scale=None,

native_scale=False,

formatter=None,

orient=None,

color=None,

palette=None,

hue_norm=None,

legend='auto',

legend_out=True,

sharex=True,

sharey=True,

margin_titles=False,

facet_kws=None,

ci=<deprecated>,

**kwargs
)
```

Figure-level interface for drawing categorical plots onto a FacetGrid.

This function provides access to several axes-level functions that show the relationship between a numerical and one or more categorical variables using one of several visual representations. The `kind` parameter selects the underlying axes-level function to use.

Categorical scatterplots:

- :func:`stripplot` (with `kind="strip"`; the default)
- :func:`swarmplot` (with `kind="swarm"`)

Categorical distribution plots:

- :func:`boxplot` (with `kind="box"`)
- :func:`violinplot` (with `kind="violin"`)
- :func:`boxenplot` (with `kind="boxen"`)

Categorical estimate plots:

- :func:`pointplot` (with `kind="point"`)
- :func:`barplot` (with `kind="bar"`)
- :func:`countplot` (with `kind="count"`)

Extra keyword arguments are passed to the underlying function, so you should refer to the documentation for each to see kind-specific options.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting ``native_scale=True``.

After plotting, the `:class:`FacetGrid`` with the plot is returned and can be used directly to tweak supporting plot details or add other layers.

#### Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If ``x`` and ``y`` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x, y, hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`row, col` : names of variables in ``data`` or vector data

Categorical variables that will determine the faceting of the grid.

`kind` : str

The kind of plot to draw, corresponds to the name of a categorical axes-level plotting function. Options are: "strip", "swarm", "box", "violin", "boxen", "point", "bar", or "count".

`estimator` : string or callable that maps vector -> scalar

Statistical function to estimate within each categorical bin.

`errorbar` : string, (string, number) tuple, callable or None

Name of errorbar method (either "ci", "pi", "se", or "sd"), or a tuple

with a method name and a level parameter, or a function that maps from a vector to a (min, max) interval, or None to hide errorbar. See the `:doc:`errorbar tutorial </tutorial/error_bars>`` for more information.

.. versionadded:: v0.12.0

n\_boot : int

Number of bootstrap samples used to compute confidence intervals.

seed : int, ``numpy.random.Generator``, or ``numpy.random.RandomState``

Seed or random number generator for reproducible bootstrapping.

units : name of variable in ``data`` or vector data

Identifier of sampling units; used by the errorbar function to perform a multilevel bootstrap and account for repeated measures

weights : name of variable in ``data`` or vector data

Data values or column used to compute weighted statistics.

Note that the use of weights may limit other statistical options.

.. versionadded:: v0.13.1

order, hue\_order : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

row\_order, col\_order : lists of strings

Order to organize the rows and/or columns of the grid in; otherwise the orders are inferred from the data objects.

col\_wrap : int

"Wrap" the column variable at this width, so that the column facets span multiple rows. Incompatible with a ``row`` facet.

height : scalar

Height (in inches) of each facet. See also: ``aspect``.

aspect : scalar

Aspect ratio of each facet, so that ``aspect \* height`` gives the width of each facet in inches.

native\_scale : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

formatter : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

orient : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both `x` and `y` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added 'x'/'y' as options, equivalent to 'v'/'h'.

color : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a



dictionary mapping hue levels to matplotlib colors.

hue\_norm : tuple or :class:`matplotlib.colors.Normalize` object

Normalization in data units for colormap applied to the `hue`

variable when it is numeric. Not relevant if `hue` is categorical.

.. versionadded:: v0.12.0

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric `hue` and `size`

variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto",

choose between brief or full representation based on number of levels.

If `False`, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

legend\_out : bool

If ``True``, the figure size will be extended, and the legend will be

drawn outside the plot on the center right.

share{x,y} : bool, 'col', or 'row' optional

If true, the facets will share y axes across columns and/or x axes

across rows.

margin\_titles : bool

If ``True``, the titles for the row variable are drawn to the right of

the last column. This option is experimental and may not work in all

cases.

facet\_kws : dict

Dictionary of other keyword arguments to pass to :class:`FacetGrid`.

kwargs : key, value pairings

Other keyword arguments are passed through to the underlying plotting function.

## Returns

-----

:class:`FacetGrid`

Returns the :class:`FacetGrid` object with the plot on it for further tweaking.

## Examples

-----

```
.. include:: ../docstrings/catplot.rst
```

```
countplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 order=None,
 hue_order=None,
 orient=None,
 color=None,
 palette=None,
 saturation=0.75,
 fill=True,
 hue_norm=None,
```

```
stat='count',

width=0.8,

dodge='auto',

gap=0,

log_scale=None,

native_scale=False,

formatter=None,

legend='auto',

ax=None,

**kwargs

)
```

Show the counts of observations in each categorical bin using bars.

A count plot can be thought of as a histogram across a categorical, instead of quantitative, variable. The basic API and options are identical to those for `:func:`barplot``, so you can compare counts across nested variables.

Note that `:func:`histplot`` function offers similar functionality with additional features (e.g. bar stacking), although its default behavior is somewhat different.

See the `:ref:`tutorial <category_tutorial>`` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting ``native_scale=True``.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If ``x`` and ``y`` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x`, `y`, `hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order`, `hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both ``x`` and ``y`` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added `'x'/'y'` as options, equivalent to `'v'/'h'`.

`color` : matplotlib color

Single color for the elements in the plot.

`palette` : palette name, list, or dict

Colors to use for the different levels of the ```hue``` variable. Should be something that can be interpreted by `:func:`color_palette``, or a dictionary mapping hue levels to matplotlib colors.

`saturation` : float

Proportion of the original saturation to draw fill colors in. Large

patches often look better with desaturated colors, but set this to

``1`` if you want the colors to perfectly match the input values.

`hue_norm` : tuple or `:class:`matplotlib.colors.Normalize`` object

Normalization in data units for colormap applied to the ``hue``

variable when it is numeric. Not relevant if ``hue`` is categorical.

.. versionadded:: v0.12.0

`stat` : {'count', 'percent', 'proportion', 'probability'}

Statistic to compute; when not ``'count'``, bar heights will be normalized so that they sum to 100 (for ``'percent'``) or 1 (otherwise) across the plot.

.. versionadded:: v0.13.0

`width` : float

Width allotted to each element on the orient axis. When ``native_scale=True``,

it is relative to the minimum distance between two values in the native scale.

`dodge` : "auto" or bool

When hue mapping is used, whether elements should be narrowed and shifted along the orient axis to eliminate overlap. If ``"auto"``, set to ``True`` when the orient variable is crossed with the categorical variable or ``False`` otherwise.

.. versionchanged:: 0.13.0

Added ``"auto"`` mode as a new default.

`log_scale` : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When ``None`` or ``False``, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

`native_scale` : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

`formatter` : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

`legend` : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

`ax` : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

`kwargs` : key, value mappings

Other parameters are passed through to `:class:`matplotlib.patches.Rectangle``.

Returns

-----

ax : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

See Also

-----

histplot : Bin and count observations with additional options.

barplot : Show point estimates and confidence intervals using bars.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

Examples

-----

```
.. include:: ../docstrings/countplot.rst
```

pointplot(

data=None,

\*,

x=None,

y=None,

hue=None,

order=None,

hue\_order=None,

estimator='mean',

errorbar=('ci', 95),

n\_boot=1000,

seed=None,

units=None,

```
weights=None,

color=None,

palette=None,

hue_norm=None,

markers=<default>,

linestyles=<default>,

dodge=False,

log_scale=None,

native_scale=False,

orient=None,

cappedsize=0,

formatter=None,

legend='auto',

err_kws=None,

ci=<deprecated>,

errwidth=<deprecated>,

join=<deprecated>,

scale=<deprecated>,

ax=None,

**kwargs
)
```

Show point estimates and errors using lines with markers.

A point plot represents an estimate of central tendency for a numeric variable by the position of the dot and provides some indication of the uncertainty around that estimate using error bars.



Point plots can be more useful than bar plots for focusing comparisons between different levels of one or more categorical variables. They are particularly adept at showing interactions: how the relationship between levels of one categorical variable changes across levels of a second categorical variable. The lines that join each point from the same `hue` level allow interactions to be judged by differences in slope, which is easier for the eyes than comparing the heights of several groups of points or bars.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis. As of version 0.13.0, this can be disabled by setting `native_scale=True`.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If `x` and `y` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x`, `y`, `hue` : names of variables in `data` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order`, `hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

estimator : string or callable that maps vector -> scalar

Statistical function to estimate within each categorical bin.

errorbar : string, (string, number) tuple, callable or None

Name of errorbar method (either "ci", "pi", "se", or "sd"), or a tuple with a method name and a level parameter, or a function that maps from a vector to a (min, max) interval, or None to hide errorbar. See the :doc:`errorbar tutorial </tutorial/errorBars>` for more information.

.. versionadded:: v0.12.0

n\_boot : int

Number of bootstrap samples used to compute confidence intervals.

seed : int, `numpy.random.Generator`, or `numpy.random.RandomState`

Seed or random number generator for reproducible bootstrapping.

units : name of variable in `data` or vector data

Identifier of sampling units; used by the errorbar function to perform a multilevel bootstrap and account for repeated measures

weights : name of variable in `data` or vector data

Data values or column used to compute weighted statistics.

Note that the use of weights may limit other statistical options.

.. versionadded:: v0.13.1

color : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

markers : string or list of strings

Markers to use for each of the `hue` levels.

linestyles : string or list of strings

Line styles to use for each of the `hue` levels.

dodge : bool or float

Amount to separate the points for each level of the `hue` variable along the categorical axis. Setting to `True` will apply a small default.

log\_scale : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When `None` or `False`, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

native\_scale : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

orient : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both `x` and `y` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added 'x'/'y' as options, equivalent to 'v'/'h'.

capsize : float

Width of the "caps" on error bars, relative to bar spacing.

formatter : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

err\_kws : dict

Parameters of `:class:`matplotlib.lines.Line2D``, for the error bar artists.

.. versionadded:: v0.13.0

ci : float

Level of the confidence interval to show, in `[0, 100]`.

.. deprecated:: v0.12.0

Use ``errorbar=("ci", ...)``.

errwidth : float

Thickness of error bar lines (and caps), in points.

.. deprecated:: 0.13.0

Use ``err_kws={'linewidth': ...}``.

join : bool

If ``True``, connect point estimates with a line.

.. deprecated:: v0.13.0

Set ``linestyle="none"`` to remove the lines between the points.

scale : float

Scale factor for the plot elements.

.. deprecated:: v0.13.0

Control element sizes with `:class:`matplotlib.lines.Line2D`` parameters.

ax : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

kwargs : key, value mappings

Other parameters are passed through to `:class:`matplotlib.lines.Line2D``.

.. versionadded:: v0.13.0

Returns

-----

ax : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

See Also

-----

barplot : Show point estimates and confidence intervals using bars.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

## Notes

-----

It is important to keep in mind that a point plot shows only the mean (or other estimator) value, but in many cases it may be more informative to show the distribution of values at each level of the categorical variables. In that case, other approaches such as a box or violin plot may be more appropriate.

## Examples

-----

```
.. include:: ../docstrings/pointplot.rst
```

```
stripplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 order=None,
 hue_order=None,
 jitter=True,
 dodge=False,
 orient=None,
 color=None,
 palette=None,
```

```
size=5,

edgecolor=<default>,

linewidth=0,

hue_norm=None,

log_scale=None,

native_scale=False,

formatter=None,

legend='auto',

ax=None,

**kwargs
)
```

Draw a categorical scatterplot using jitter to reduce overplotting.

A strip plot can be drawn on its own, but it is also a good complement to a box or violin plot in cases where you want to show all observations along with some representation of the underlying distribution.

See the :ref:`tutorial <categorical\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis.

As of version 0.13.0, this can be disabled by setting ``native_scale=True``.

Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If ``x`` and ``y`` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x`, `y`, `hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order`, `hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`jitter` : float, ``True`/`1`` is special-cased

Amount of jitter (only along the categorical axis) to apply. This can be useful when you have many points and they overlap, so that it is easier to see the distribution. You can specify the amount of jitter (half the width of the uniform random variable support), or use ``True`` for a good default.

`dodge` : bool

When a ``hue`` variable is assigned, setting this to ``True`` will separate the strips for different hue levels along the categorical axis and narrow the amount of space allotted to each strip. Otherwise, the points for each level will be plotted in the same strip.

`orient` : `"v"` | `"h"` | `"x"` | `"y"`

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both ``x`` and ``y`` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added `'x'/'y'` as options, equivalent to `'v'/'h'`.



`color : matplotlib color`

Single color for the elements in the plot.

`palette : palette name, list, or dict`

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by `:func:`color_palette``, or a dictionary mapping hue levels to matplotlib colors.

`size : float`

Radius of the markers, in points.

`edgecolor : matplotlib color, "gray" is special-cased`

Color of the lines around each point. If you pass `"gray"`, the brightness is determined by the color palette used for the body of the points. Note that ``stripplot`` has ``linewidth=0`` by default, so edge colors are only visible with nonzero line width.

`linewidth : float`

Width of the lines that frame the plot elements.

`hue_norm : tuple or :class:`matplotlib.colors.Normalize` object`

Normalization in data units for colormap applied to the ``hue`` variable when it is numeric. Not relevant if ``hue`` is categorical.

.. versionadded:: v0.12.0

`log_scale : bool or number, or pair of bools or numbers`

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently. Numeric values are interpreted as the desired base (default 10). When ``None`` or ``False``, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

`native_scale : bool`

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

`formatter : callable`

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

`legend : "auto", "brief", "full", or False`

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

`ax : matplotlib Axes`

Axes object to draw the plot onto, otherwise uses the current Axes.

`kwargs : key, value mappings`

Other keyword arguments are passed through to

`:meth:`matplotlib.axes.Axes.scatter`.`

Returns

-----

`ax : matplotlib Axes`

Returns the Axes object with the plot drawn onto it.

See Also

-----

swarmplot : A categorical scatterplot where the points do not overlap. Can  
be used with other plots to show each observation.

boxplot : A traditional box-and-whisker plot with a similar API.

violinplot : A combination of boxplot and kernel density estimation.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

Examples

-----

```
.. include:: ../docstrings/stripplot.rst
```

```
swarmplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 order=None,

 hue_order=None,

 dodge=False,

 orient=None,

 color=None,

 palette=None,

 size=5,
```

```
edgecolor=None,

linewidth=0,

hue_norm=None,

log_scale=None,

native_scale=False,

formatter=None,

legend='auto',

warn_thresh=0.05,

ax=None,

**kwargs
)
```

Draw a categorical scatterplot with points adjusted to be non-overlapping.

This function is similar to :func:`stripplot`, but the points are adjusted (only along the categorical axis) so that they don't overlap. This gives a better representation of the distribution of values, but it does not scale well to large numbers of observations. This style of plot is sometimes called a "beeswarm".

A swarm plot can be drawn on its own, but it is also a good complement to a box or violin plot in cases where you want to show all observations along with some representation of the underlying distribution.

See the :ref:`tutorial <categorical\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical

and draws data at ordinal positions (0, 1, ... n) on the relevant axis.

As of version 0.13.0, this can be disabled by setting ``native_scale=True``.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If ``x`` and ``y`` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x, y, hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order, hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`dodge` : bool

When a ``hue`` variable is assigned, setting this to ``True`` will separate the swarms for different hue levels along the categorical axis and narrow the amount of space allotted to each strip. Otherwise, the points for each level will be plotted in the same swarm.

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both ``x`` and ``y`` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added `'x'/'y'` as options, equivalent to `'v'/'h'`.

color : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

size : float

Radius of the markers, in points.

edgecolor : matplotlib color, "gray" is special-cased

Color of the lines around each point. If you pass ``"gray"`, the brightness is determined by the color palette used for the body of the points.

linewidth : float

Width of the lines that frame the plot elements.

log\_scale : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently. Numeric values are interpreted as the desired base (default 10). When ``None`` or ``False``, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

native\_scale : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

formatter : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

ax : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

kwargs : key, value mappings

Other keyword arguments are passed through to

`:meth:`matplotlib.axes.Axes.scatter``.

Returns

-----

ax : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

See Also

-----

boxplot : A traditional box-and-whisker plot with a similar API.

violinplot : A combination of boxplot and kernel density estimation.

stripplot : A scatterplot where one variable is categorical. Can be used

in conjunction with other plots to show each observation.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

## Examples

-----

```
.. include:: ../docstrings/swarmplot.rst
```

```
violinplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 order=None,

 hue_order=None,

 orient=None,

 color=None,

 palette=None,

 saturation=0.75,

 fill=True,

 inner='box',

 split=False,

 width=0.8,

 dodge='auto',

 gap=0,

 linewidth=None,
```



```
linecolor='auto',

cut=2,

gridsize=100,

bw_method='scott',

bw_adjust=1,

density_norm='area',

common_norm=False,

hue_norm=None,

formatter=None,

log_scale=None,

native_scale=False,

legend='auto',

scale=<deprecated>,

scale_hue=<deprecated>,

bw=<deprecated>,

inner_kws=None,

ax=None,

**kwargs

)
```

Draw a patch representing a KDE and add observations or box plot statistics.

A violin plot plays a similar role as a box-and-whisker plot. It shows the distribution of data points after grouping by one (or more) variables.

Unlike a box plot, each violin is drawn using a kernel density estimate of the underlying distribution.

See the :ref:`tutorial <category\_tutorial>` for more information.

.. note::

By default, this function treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis.

As of version 0.13.0, this can be disabled by setting ``native_scale=True``.

## Parameters

-----

`data` : DataFrame, Series, dict, array, or list of arrays

Dataset for plotting. If ``x`` and ``y`` are absent, this is interpreted as wide-form. Otherwise it is expected to be long-form.

`x, y, hue` : names of variables in ``data`` or vector data

Inputs for plotting long-form data. See examples for interpretation.

`order, hue_order` : lists of strings

Order to plot the categorical levels in; otherwise the levels are inferred from the data objects.

`orient` : "v" | "h" | "x" | "y"

Orientation of the plot (vertical or horizontal). This is usually inferred based on the type of the input variables, but it can be used to resolve ambiguity when both ``x`` and ``y`` are numeric or when plotting wide-form data.

.. versionchanged:: v0.13.0

Added `'x'/'y'` as options, equivalent to `'v'/'h'`.

`color` : matplotlib color

Single color for the elements in the plot.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

saturation : float

Proportion of the original saturation to draw fill colors in. Large patches often look better with desaturated colors, but set this to `1` if you want the colors to perfectly match the input values.

fill : bool

If True, use a solid patch. Otherwise, draw as line art.

.. versionadded:: v0.13.0

inner : {"box", "quart", "point", "stick", None}

Representation of the data in the violin interior. One of the following:

- ``"box"``: draw a miniature box-and-whisker plot
- ``"quart"``: show the quartiles of the data
- ``"point"`` or ``"stick"``: show each observation

split : bool

Show an un-mirrored distribution, alternating sides when using `hue`.

.. versionchanged:: v0.13.0

Previously, this option required a `hue` variable with exactly two levels.

width : float

Width allotted to each element on the orient axis. When `native\_scale=True`, it is relative to the minimum distance between two values in the native scale.

dodge : "auto" or bool

When hue mapping is used, whether elements should be narrowed and shifted along the orient axis to eliminate overlap. If `"auto"`, set to `True` when the orient variable is crossed with the categorical variable or `False` otherwise.

.. versionchanged:: 0.13.0

Added `"auto"` mode as a new default.

gap : float

Shrink on the orient axis by this factor to add a gap between dodged elements.

.. versionadded:: 0.13.0

linewidth : float

Width of the lines that frame the plot elements.

linecolor : color

Color to use for line elements, when `fill` is True.

.. versionadded:: v0.13.0

cut : float

Distance, in units of bandwidth, to extend the density past extreme datapoints. Set to 0 to limit the violin within the data range.

gridsize : int

Number of points in the discrete grid used to evaluate the KDE.

bw\_method : {"scott", "silverman", float}

Either the name of a reference rule or the scale factor to use when computing the kernel bandwidth. The actual kernel size will be determined by multiplying the scale factor by the standard deviation of the data within each group.

.. versionadded:: v0.13.0

`bw_adjust`: float

Factor that scales the bandwidth to use more or less smoothing.

.. versionadded:: v0.13.0

`density_norm` : {"area", "count", "width"}

Method that normalizes each density to determine the violin's width.

If ``area``, each violin will have the same area. If ``count``, the width will be proportional to the number of observations. If ``width``, each violin will have the same width.

.. versionadded:: v0.13.0

`common_norm` : bool

When ``True``, normalize the density across all violins.

.. versionadded:: v0.13.0

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize` object

Normalization in data units for colormap applied to the ``hue`` variable when it is numeric. Not relevant if ``hue`` is categorical.

.. versionadded:: v0.12.0

`formatter` : callable

Function for converting categorical data into strings. Affects both grouping and tick labels.

.. versionadded:: v0.13.0

`log_scale` : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When ``None`` or ``False``, seaborn defers to the existing Axes scale.

.. versionadded:: v0.13.0

`native_scale` : bool

When True, numeric or datetime values on the categorical axis will maintain their original scaling rather than being converted to fixed indices.

.. versionadded:: v0.13.0

`legend` : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

.. versionadded:: v0.13.0

`scale` : {"area", "count", "width"}

.. deprecated:: v0.13.0

See ``density_norm``.

`scale_hue` : bool

.. deprecated:: v0.13.0

See ``common_norm``.

`bw` : {'scott', 'silverman', float}

.. deprecated:: v0.13.0

See ``bw_method`` and ``bw_adjust``.

inner\_kws : dict of key, value mappings

Keyword arguments for the "inner" plot, passed to one of:

- :class:`matplotlib.collections.LineCollection` (with ``inner="stick"``)
- :meth:`matplotlib.axes.Axes.scatter` (with ``inner="point"``)
- :meth:`matplotlib.axes.Axes.plot` (with ``inner="quart"`` or ``inner="box"``)

Additionally, with ``inner="box"``, the keywords ``box_width``, ``whis_width``, and ``marker`` receive special handling for the components of the "box" plot.

.. versionadded:: v0.13.0

ax : matplotlib Axes

Axes object to draw the plot onto, otherwise uses the current Axes.

kwargs : key, value mappings

Keyword arguments for the violin patches, passed through to

:meth:`matplotlib.axes.Axes.fill\_between`.

Returns

-----

ax : matplotlib Axes

Returns the Axes object with the plot drawn onto it.

See Also

-----

boxplot : A traditional box-and-whisker plot with a similar API.

stripplot : A scatterplot where one variable is categorical. Can be used

in conjunction with other plots to show each observation.

swarmplot : A categorical scatterplot where the points do not overlap. Can

be used with other plots to show each observation.

catplot : Combine a categorical plot with a :class:`FacetGrid`.

## Examples

-----

```
.. include:: ../docstrings/violinplot.rst
```

## DATA

```
__all__ = ['catplot', 'stripplot', 'swarmplot', 'boxplot', 'violinplot...]
```

## FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\categorical.py

Help on module seaborn.cm in seaborn:

## NAME

seaborn.cm

## DATA

```
crest = <matplotlib.colors.ListedColormap object>
```

```
crest_r = <matplotlib.colors.ListedColormap object>
```

```
flare = <matplotlib.colors.ListedColormap object>
```

```
flare_r = <matplotlib.colors.ListedColormap object>
```



```
icefire = <matplotlib.colors.ListedColormap object>

icefire_r = <matplotlib.colors.ListedColormap object>

mako = <matplotlib.colors.ListedColormap object>

mako_r = <matplotlib.colors.ListedColormap object>

rocket = <matplotlib.colors.ListedColormap object>

rocket_r = <matplotlib.colors.ListedColormap object>

vlag = <matplotlib.colors.ListedColormap object>

vlag_r = <matplotlib.colors.ListedColormap object>
```

#### FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\cm.py
```

Help on package seaborn.colors in seaborn:

#### NAME

```
seaborn.colors
```

#### PACKAGE CONTENTS

```
crayons
xkcd_rgb
```

#### DATA

```
crayons = {'Almond': '#EFDECD', 'Antique Brass': '#CD9575', 'Apricot':...

xkcd_rgb = {'acid green': '#8ffe09', 'adobe': '#bd6c48', 'algae': '#54...
```

#### FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\colors__init__.py
```

Help on module seaborn.distributions in seaborn:

#### NAME

seaborn.distributions - Plotting functions for visualizing distributions.

#### FUNCTIONS

```
displot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 row=None,
 col=None,
 weights=None,
 kind='hist',
 rug=False,
 rug_kws=None,
 log_scale=None,
 legend=True,
 palette=None,
 hue_order=None,
 hue_norm=None,
```

```
color=None,

col_wrap=None,

row_order=None,

col_order=None,

height=5,

aspect=1,

facet_kws=None,

**kwargs
)
```

Figure-level interface for drawing distribution plots onto a FacetGrid.

This function provides access to several approaches for visualizing the univariate or bivariate distribution of data, including subsets of data defined by semantic mapping and faceting across multiple subplots. The ``kind`` parameter selects the approach to use:

- :func:`histplot` (with ``kind="hist"``; the default)
- :func:`kdeplot` (with ``kind="kde"``)
- :func:`ecdfplot` (with ``kind="ecdf"``; univariate-only)

Additionally, a :func:`rugplot` can be added to any kind of plot to show individual observations.

Extra keyword arguments are passed to the underlying function, so you should refer to the documentation for each to understand the complete set of options for making plots with this interface.

See the :doc:`distribution plots tutorial <../tutorial/distributions>` for a more in-depth discussion of the relative strengths and weaknesses of each approach. The distinction between figure-level and axes-level functions is explained further in the :doc:`user guide <../tutorial/function\_overview>`.

## Parameters

-----

**data** : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

**x, y** : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

**hue** : vector or key in ``data``

Semantic variable that is mapped to determine the color of plot elements.

**row, col** : vectors or keys in ``data``

Variables that define subsets to plot on different facets.

**weights** : vector or key in ``data``

Observation weights used for computing the distribution function.

**kind** : {"hist", "kde", "ecdf"}

Approach for visualizing the data. Selects the underlying plotting function and determines the additional set of valid parameters.

**rug** : bool

If True, show each observation with marginal ticks (as in :func:`rugplot`).

**rug\_kws** : dict

Parameters to control the appearance of the rug plot.

**log\_scale** : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently. Numeric values are interpreted as the desired base (default 10). When `None` or `False`, seaborn defers to the existing Axes scale.

legend : bool

If False, suppress the legend for semantic variables.

palette : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to :func:`color\_palette`. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

hue\_order : vector of strings

Specify the order of processing and plotting for categorical levels of the ``hue`` semantic.

hue\_norm : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units or an object that will map from data units into a [0, 1] interval. Usage implies numeric mapping.

color : :mod:`matplotlib color` <matplotlib.colors>

Single color specification for when hue mapping is not used. Otherwise, the plot will try to hook into the matplotlib property cycle.

col\_wrap : int

"Wrap" the column variable at this width, so that the column facets span multiple rows. Incompatible with a ``row`` facet.

{row,col}\_order : vector of strings

Specify the order in which levels of the ``row`` and/or ``col`` variables appear in the grid of subplots.

height : scalar

Height (in inches) of each facet. See also: ``aspect``.

aspect : scalar

Aspect ratio of each facet, so that ``aspect \* height`` gives the width of each facet in inches.

facet\_kws : dict

Additional parameters passed to :class:`FacetGrid`.

kwargs

Other keyword arguments are documented with the relevant axes-level function:

- :func:`histplot` (with ``kind="hist"``)
- :func:`kdeplot` (with ``kind="kde"``)
- :func:`ecdfplot` (with ``kind="ecdf"``)

Returns

-----

:class:`FacetGrid`

An object managing one or more subplots that correspond to conditional data subsets with convenient methods for batch-setting of axes attributes.

See Also

-----

histplot : Plot a histogram of binned counts with optional normalization or smoothing.

kdeplot : Plot univariate or bivariate distributions using kernel density estimation.

rugplot : Plot a tick at each observation value along the x and/or y axes.

ecdfplot : Plot empirical cumulative distribution functions.

jointplot : Draw a bivariate plot with univariate marginal distributions.

## Examples

-----

See the API documentation for the axes-level functions for more details about the breadth of options available for each plot kind.

```
.. include:: ../docstrings/displot.rst
```

```
distplot(
 a=None,
 bins=None,
 hist=True,
 kde=True,
 rug=False,
 fit=None,
 hist_kws=None,
 kde_kws=None,
 rug_kws=None,
 fit_kws=None,
 color=None,
 vertical=False,
 norm_hist=False,
 axlabel=None,
 label=None,
 ax=None,
 x=None
```

)

DEPRECATED

This function has been deprecated and will be removed in seaborn v0.14.0.

It has been replaced by :func:`histplot` and :func:`displot`, two functions with a modern API and many more capabilities.

For a guide to updating, please see this notebook:

<https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
ecdfplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 weights=None,
 stat='proportion',
 complementary=False,
 palette=None,
 hue_order=None,
 hue_norm=None,
 log_scale=None,
 legend=True,
 ax=None,
 **kwargs
```



)

Plot empirical cumulative distribution functions.

An ECDF represents the proportion or count of observations falling below each unique value in a dataset. Compared to a histogram or density plot, it has the advantage that each observation is visualized directly, meaning that there are no binning or smoothing parameters that need to be adjusted. It also aids direct comparisons between multiple distributions. A downside is that the relationship between the appearance of the plot and the basic properties of the distribution (such as its central tendency, variance, and the presence of any bimodality) may not be as intuitive.

More information is provided in the :ref:`user guide <tutorial\_ecdf>`.

Parameters

-----

**data** : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

**x, y** : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

**hue** : vector or key in ``data``

Semantic variable that is mapped to determine the color of plot elements.

**weights** : vector or key in ``data``

If provided, weight the contribution of the corresponding data points towards the cumulative distribution using these values.

`stat : {"proportion", "percent", "count"}}`

Distribution statistic to compute.

`complementary : bool`

If True, use the complementary CDF ( $1 - \text{CDF}$ )

`palette : string, list, dict, or :class:`matplotlib.colors.Colormap``

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to `:func:`color_palette``. List or dict values

imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order : vector of strings`

Specify the order of processing and plotting for categorical levels of the

``hue`` semantic.

`hue_norm : tuple or :class:`matplotlib.colors.Normalize``

Either a pair of values that set the normalization range in data units

or an object that will map from data units into a  $[0, 1]$  interval. Usage

implies numeric mapping.

`log_scale : bool or number, or pair of bools or numbers`

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When `None` or `False`, seaborn defers to the existing Axes scale.

`legend : bool`

If False, suppress the legend for semantic variables.

`ax : :class:`matplotlib.axes.Axes``

Pre-existing axes for the plot. Otherwise, call `:func:`matplotlib.pyplot.gca`` internally.

`kwargs`

Other keyword arguments are passed to `:meth:`matplotlib.axes.Axes.plot``.

## Returns

-----

:class:`matplotlib.axes.Axes`

The matplotlib axes containing the plot.

## See Also

-----

`displot` : Figure-level interface to distribution plot functions.

`histplot` : Plot a histogram of binned counts with optional normalization or smoothing.

`kdeplot` : Plot univariate or bivariate distributions using kernel density estimation.

`rugplot` : Plot a tick at each observation value along the x and/or y axes.

## Examples

-----

```
.. include:: ../docstrings/ecdfplot.rst
```

```
histplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 weights=None,

 stat='count',

 bins='auto',
```

```
binwidth=None,

binrange=None,

discrete=None,

cumulative=False,

common_bins=True,

common_norm=True,

multiple='layer',

element='bars',

fill=True,

shrink=1,

kde=False,

kde_kws=None,

line_kws=None,

thresh=0,

pthresh=None,

pmax=None,

cbar=False,

cbar_ax=None,

cbar_kws=None,

palette=None,

hue_order=None,

hue_norm=None,

color=None,

log_scale=None,

legend=True,

ax=None,

**kwargs
```

)

Plot univariate or bivariate histograms to show distributions of datasets.

A histogram is a classic visualization tool that represents the distribution of one or more variables by counting the number of observations that fall within discrete bins.

This function can normalize the statistic computed within each bin to estimate frequency, density or probability mass, and it can add a smooth curve obtained using a kernel density estimate, similar to `:func:`kdeplot``.

More information is provided in the `:ref:`user guide <tutorial_hist>``.

#### Parameters

-----

`data` : `:class:`pandas.DataFrame``, `:class:`numpy.ndarray``, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

`x, y` : vectors or keys in ```data```

Variables that specify positions on the x and y axes.

`hue` : vector or key in ```data```

Semantic variable that is mapped to determine the color of plot elements.

`weights` : vector or key in ```data```

If provided, weight the contribution of the corresponding data points towards the count in each bin by these factors.

`stat` : str

Aggregate statistic to compute in each bin.

- ``count``: show the number of observations in each bin
- ``frequency``: show the number of observations divided by the bin width
- ``probability`` or ``proportion``: normalize such that bar heights sum to 1
- ``percent``: normalize such that bar heights sum to 100
- ``density``: normalize such that the total area of the histogram equals 1

`bins` : str, number, vector, or a pair of such values

Generic bin parameter that can be the name of a reference rule,  
the number of bins, or the breaks of the bins.

Passed to `:func:`numpy.histogram_bin_edges``.

`binwidth` : number or pair of numbers

Width of each bin, overrides ``bins`` but can be used with  
``binrange``.

`binrange` : pair of numbers or a pair of pairs

Lowest and highest value for bin edges; can be used either  
with ``bins`` or ``binwidth``. Defaults to data extremes.

`discrete` : bool

If True, default to ``binwidth=1`` and draw the bars so that they are  
centered on their corresponding data points. This avoids "gaps" that may  
otherwise appear when using discrete (integer) data.

`cumulative` : bool

If True, plot the cumulative counts as bins increase.

`common_bins` : bool

If True, use the same bins when semantic variables produce multiple  
plots. If using a reference rule to determine the bins, it will be computed  
with the full dataset.

`common_norm : bool`

If True and using a normalized statistic, the normalization will apply over the full dataset. Otherwise, normalize each histogram independently.

`multiple : {"layer", "dodge", "stack", "fill"}`

Approach to resolving multiple elements when semantic mapping creates subsets.

Only relevant with univariate data.

`element : {"bars", "step", "poly"}`

Visual representation of the histogram statistic.

Only relevant with univariate data.

`fill : bool`

If True, fill in the space under the histogram.

Only relevant with univariate data.

`shrink : number`

Scale the width of each bar relative to the binwidth by this factor.

Only relevant with univariate data.

`kde : bool`

If True, compute a kernel density estimate to smooth the distribution and show on the plot as (one or more) line(s).

Only relevant with univariate data.

`kde_kws : dict`

Parameters that control the KDE computation, as in `:func:`kdeplot``.

`line_kws : dict`

Parameters that control the KDE visualization, passed to `:meth:`matplotlib.axes.Axes.plot``.

`thresh : number or None`

Cells with a statistic less than or equal to this value will be transparent.

Only relevant with bivariate data.

`pthresh` : number or None

Like `thresh`, but a value in `[0, 1]` such that cells with aggregate counts (or other statistics, when used) up to this proportion of the total will be transparent.

`pmax` : number or None

A value in `[0, 1]` that sets that saturation point for the colormap at a value such that cells below constitute this proportion of the total count (or other statistic, when used).

`cbar` : bool

If True, add a colorbar to annotate the color mapping in a bivariate plot.

Note: Does not currently support plots with a `hue` variable well.

`cbar_ax` : :class:`matplotlib.axes.Axes`

Pre-existing axes for the colorbar.

`cbar_kws` : dict

Additional parameters passed to :meth:`matplotlib.figure.Figure.colorbar`.

`palette` : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the `hue` semantic.

String values are passed to :func:`color\_palette`. List or dict values

imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order` : vector of strings

Specify the order of processing and plotting for categorical levels of the `hue` semantic.

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units

or an object that will map from data units into a `[0, 1]` interval. Usage

implies numeric mapping.

`color` : :mod:`matplotlib color <matplotlib.colors>`



Single color specification for when hue mapping is not used. Otherwise, the plot will try to hook into the matplotlib property cycle.

`log_scale` : bool or number, or pair of bools or numbers

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When ``None`` or ``False``, seaborn defers to the existing Axes scale.

`legend` : bool

If False, suppress the legend for semantic variables.

`ax` : :class:`matplotlib.axes.Axes`

Pre-existing axes for the plot. Otherwise, call :func:`matplotlib.pyplot.gca` internally.

`kwargs`

Other keyword arguments are passed to one of the following matplotlib functions:

- :meth:`matplotlib.axes.Axes.bar` (univariate, element="bars")
- :meth:`matplotlib.axes.Axes.fill\_between` (univariate, other element, fill=True)
- :meth:`matplotlib.axes.Axes.plot` (univariate, other element, fill=False)
- :meth:`matplotlib.axes.Axes.pcolormesh` (bivariate)

Returns

-----

:class:`matplotlib.axes.Axes`

The matplotlib axes containing the plot.

See Also

-----

displot : Figure-level interface to distribution plot functions.

kdeplot : Plot univariate or bivariate distributions using kernel density estimation.

rugplot : Plot a tick at each observation value along the x and/or y axes.

ecdfplot : Plot empirical cumulative distribution functions.

jointplot : Draw a bivariate plot with univariate marginal distributions.

## Notes

-----

The choice of bins for computing and plotting a histogram can exert substantial influence on the insights that one is able to draw from the visualization. If the bins are too large, they may erase important features. On the other hand, bins that are too small may be dominated by random variability, obscuring the shape of the true underlying distribution. The default bin size is determined using a reference rule that depends on the sample size and variance. This works well in many cases, (i.e., with "well-behaved" data) but it fails in others. It is always a good to try different bin sizes to be sure that you are not missing something important. This function allows you to specify bins in several different ways, such as by setting the total number of bins to use, the width of each bin, or the specific locations where the bins should break.

## Examples

-----

```
.. include:: ../docstrings/histplot.rst
```

```
kdeplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 weights=None,
 palette=None,
 hue_order=None,
 hue_norm=None,
 color=None,
 fill=None,
 multiple='layer',
 common_norm=True,
 common_grid=False,
 cumulative=False,
 bw_method='scott',
 bw_adjust=1,
 warn_singular=True,
 log_scale=None,
 levels=10,
 thresh=0.05,
 gridsize=200,
 cut=3,
 clip=None,
 legend=True,
```

```
cbar=False,

cbar_ax=None,

cbar_kws=None,

ax=None,

**kwargs
)
```

Plot univariate or bivariate distributions using kernel density estimation.

A kernel density estimate (KDE) plot is a method for visualizing the distribution of observations in a dataset, analogous to a histogram. KDE represents the data using a continuous probability density curve in one or more dimensions.

The approach is explained further in the :ref:`user guide <tutorial\_kde>`.

Relative to a histogram, KDE can produce a plot that is less cluttered and more interpretable, especially when drawing multiple distributions. But it has the potential to introduce distortions if the underlying distribution is bounded or not smooth. Like a histogram, the quality of the representation also depends on the selection of good smoothing parameters.

#### Parameters

-----

data : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

`x, y` : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

`hue` : vector or key in ``data``

Semantic variable that is mapped to determine the color of plot elements.

`weights` : vector or key in ``data``

If provided, weight the kernel density estimation using these values.

`palette` : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to :func:`color\_palette`. List or dict values

imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order` : vector of strings

Specify the order of processing and plotting for categorical levels of the

``hue`` semantic.

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units

or an object that will map from data units into a [0, 1] interval. Usage

implies numeric mapping.

`color` : :mod:`matplotlib color` <matplotlib.colors>

Single color specification for when hue mapping is not used. Otherwise, the

plot will try to hook into the matplotlib property cycle.

`fill` : bool or None

If True, fill in the area under univariate density curves or between

bivariate contours. If None, the default depends on ``multiple``.

`multiple` : {"layer", "stack", "fill"}

Method for drawing multiple elements when semantic mapping creates subsets.

Only relevant with univariate data.

`common_norm` : bool

If True, scale each conditional density by the number of observations such that the total area under all densities sums to 1. Otherwise, normalize each density independently.

`common_grid : bool`

If True, use the same evaluation grid for each kernel density estimate.

Only relevant with univariate data.

`cumulative : bool, optional`

If True, estimate a cumulative distribution function. Requires `scipy`.

`bw_method : string, scalar, or callable, optional`

Method for determining the smoothing bandwidth to use; passed to

`:class:`scipy.stats.gaussian_kde``.

`bw_adjust : number, optional`

Factor that multiplicatively scales the value chosen using

```bw_method```. Increasing will make the curve smoother. See Notes.

`warn_singular : bool`

If True, issue a warning when trying to estimate the density of data

with zero variance.

`log_scale : bool or number, or pair of bools or numbers`

Set axis scale(s) to log. A single value sets the data axis for any numeric axes in the plot. A pair of values sets each axis independently.

Numeric values are interpreted as the desired base (default 10).

When ``None`` or ``False``, seaborn defers to the existing Axes scale.

`levels : int or vector`

Number of contour levels or values to draw contours at. A vector argument must have increasing values in `[0, 1]`. Levels correspond to iso-proportions of the density: e.g., 20% of the probability mass will lie below the contour drawn for 0.2. Only relevant with bivariate data.

thresh : number in [0, 1]

Lowest iso-proportion level at which to draw a contour line. Ignored when

``levels`` is a vector. Only relevant with bivariate data.

gridsize : int

Number of points on each dimension of the evaluation grid.

cut : number, optional

Factor, multiplied by the smoothing bandwidth, that determines how

far the evaluation grid extends past the extreme datapoints. When

set to 0, truncate the curve at the data limits.

clip : pair of numbers or None, or a pair of such pairs

Do not evaluate the density outside of these limits.

legend : bool

If False, suppress the legend for semantic variables.

cbar : bool

If True, add a colorbar to annotate the color mapping in a bivariate plot.

Note: Does not currently support plots with a ``hue`` variable well.

cbar_ax : :class:`matplotlib.axes.Axes`

Pre-existing axes for the colorbar.

cbar_kws : dict

Additional parameters passed to :meth:`matplotlib.figure.Figure.colorbar`.

ax : :class:`matplotlib.axes.Axes`

Pre-existing axes for the plot. Otherwise, call :func:`matplotlib.pyplot.gca`

internally.

kwargs

Other keyword arguments are passed to one of the following matplotlib

functions:

- :meth:`matplotlib.axes.Axes.plot` (univariate, ``fill=False``),
- :meth:`matplotlib.axes.Axes.fill\_between` (univariate, ``fill=True``),
- :meth:`matplotlib.axes.Axes.contour` (bivariate, ``fill=False``),
- :meth:`matplotlib.axes.contourf` (bivariate, ``fill=True``).

Returns

:class:`matplotlib.axes.Axes`

The matplotlib axes containing the plot.

See Also

displot : Figure-level interface to distribution plot functions.

histplot : Plot a histogram of binned counts with optional normalization or smoothing.

ecdfplot : Plot empirical cumulative distribution functions.

jointplot : Draw a bivariate plot with univariate marginal distributions.

violinplot : Draw an enhanced boxplot using kernel density estimation.

Notes

The **bandwidth**, or standard deviation of the smoothing kernel, is an important parameter. Misspecification of the bandwidth can produce a distorted representation of the data. Much like the choice of bin width in a histogram, an over-smoothed curve can erase true features of a distribution, while an under-smoothed curve can create false features out of random variability. The rule-of-thumb that sets the default bandwidth works

best when the true distribution is smooth, unimodal, and roughly bell-shaped. It is always a good idea to check the default behavior by using `bw_adjust` to increase or decrease the amount of smoothing.

Because the smoothing algorithm uses a Gaussian kernel, the estimated density curve can extend to values that do not make sense for a particular dataset. For example, the curve may be drawn over negative values when smoothing data that are naturally positive. The `cut` and `clip` parameters can be used to control the extent of the curve, but datasets that have many observations close to a natural boundary may be better served by a different visualization method.

Similar considerations apply when a dataset is naturally discrete or "spiky" (containing many repeated observations of the same value). Kernel density estimation will always produce a smooth curve, which would be misleading in these situations.

The units on the density axis are a common source of confusion. While kernel density estimation produces a probability distribution, the height of the curve at each point gives a density, not a probability. A probability can be obtained only by integrating the density across a range. The curve is normalized so that the integral over all possible values is 1, meaning that the scale of the density axis depends on the data values.

Examples

```
.. include:: ../docstrings/kdeplot.rst
```

```
rugplot(  
  
    data=None,  
  
    *,  
  
    x=None,  
  
    y=None,  
  
    hue=None,  
  
    height=0.025,  
  
    expand_margins=True,  
  
    palette=None,  
  
    hue_order=None,  
  
    hue_norm=None,  
  
    legend=True,  
  
    ax=None,  
  
    **kwargs  
)
```

Plot marginal distributions by drawing ticks along the x and y axes.

This function is intended to complement other plots by showing the location of individual observations in an unobtrusive way.

Parameters

data : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally

reshaped.

`x, y : vectors or keys in ``data```

Variables that specify positions on the x and y axes.

`hue : vector or key in ``data```

Semantic variable that is mapped to determine the color of plot elements.

`height : float`

Proportion of axes extent covered by each rug element. Can be negative.

`expand_margins : bool`

If True, increase the axes margins by the height of the rug to avoid overlap with other elements.

`palette : string, list, dict, or :class:`matplotlib.colors.Colormap``

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to `:func:`color_palette``. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order : vector of strings`

Specify the order of processing and plotting for categorical levels of the ``hue`` semantic.

`hue_norm : tuple or :class:`matplotlib.colors.Normalize``

Either a pair of values that set the normalization range in data units or an object that will map from data units into a `[0, 1]` interval. Usage implies numeric mapping.

`legend : bool`

If False, do not add a legend for semantic variables.

`ax : :class:`matplotlib.axes.Axes``

Pre-existing axes for the plot. Otherwise, call `:func:`matplotlib.pyplot.gca`` internally.

`kwargs`

Other keyword arguments are passed to

:meth:`matplotlib.collections.LineCollection`

Returns

:class:`matplotlib.axes.Axes`

The matplotlib axes containing the plot.

Examples

.. include:: ../docstrings/rugplot.rst

DATA

__all__ = ['displot', 'histplot', 'kdeplot', 'ecdfplot', 'rugplot', 'd...

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\distributions.py

Help on package seaborn.external in seaborn:

NAME

seaborn.external

PACKAGE CONTENTS

appdirs

docscrape

husl

kde

version

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\external__init__.py

Help on module seaborn.matrix in seaborn:

NAME

seaborn.matrix - Functions to visualize matrices of data.

FUNCTIONS

```
clustermap(  
    data,  
    *,  
    pivot_kws=None,  
    method='average',  
    metric='euclidean',  
    z_score=None,  
    standard_scale=None,  
    figsize=(10, 10),  
    cbar_kws=None,  
    row_cluster=True,
```

```
col_cluster=True,  
  
row_linkage=None,  
  
col_linkage=None,  
  
row_colors=None,  
  
col_colors=None,  
  
mask=None,  
  
dendrogram_ratio=0.2,  
  
colors_ratio=0.03,  
  
cbar_pos=(0.02, 0.8, 0.05, 0.18),  
  
tree_kws=None,  
  
**kwargs  
)
```

Plot a matrix dataset as a hierarchically-clustered heatmap.

This function requires `scipy` to be available.

Parameters

`data` : 2D array-like

Rectangular data for clustering. Cannot contain NAs.

`pivot_kws` : dict, optional

If ``data`` is a tidy dataframe, can provide keyword arguments for
pivot to create a rectangular dataframe.

`method` : str, optional

Linkage method to use for calculating clusters. See

:func:`scipy.cluster.hierarchy.linkage` documentation for more
information.

`metric : str, optional`

Distance metric to use for the data. See

`:func:`scipy.spatial.distance.pdist`` documentation for more options.

To use different metrics (or methods) for rows and columns, you may

construct each linkage matrix yourself and provide them as

`{row,col}_linkage`.`

`z_score : int or None, optional`

Either 0 (rows) or 1 (columns). Whether or not to calculate z-scores

for the rows or the columns. Z scores are: $z = (x - \text{mean})/\text{std}$, so

values in each row (column) will get the mean of the row (column)

subtracted, then divided by the standard deviation of the row (column).

This ensures that each row (column) has mean of 0 and variance of 1.

`standard_scale : int or None, optional`

Either 0 (rows) or 1 (columns). Whether or not to standardize that

dimension, meaning for each row or column, subtract the minimum and

divide each by its maximum.

`figsize : tuple of (width, height), optional`

Overall size of the figure.

`cbar_kws : dict, optional`

Keyword arguments to pass to `cbar_kws`` in `:func:`heatmap``, e.g. to

add a label to the colorbar.

`{row,col}_cluster : bool, optional`

If `True``, cluster the {rows, columns}.

`{row,col}_linkage : :class:`numpy.ndarray`, optional`

Precomputed linkage matrix for the rows or columns. See

`:func:`scipy.cluster.hierarchy.linkage`` for specific formats.

`{row,col}_colors : list-like or pandas DataFrame/Series, optional`

List of colors to label for either the rows or columns. Useful to evaluate whether samples within a group are clustered together. Can use nested lists or DataFrame for multiple color levels of labeling. If given as a :class:`pandas.DataFrame` or :class:`pandas.Series`, labels for the colors are extracted from the DataFrames column names or from the name of the Series. DataFrame/Series colors are also matched to the data by their index, ensuring colors are drawn in the correct order.

mask : bool array or DataFrame, optional

If passed, data will not be shown in cells where `mask` is True.

Cells with missing values are automatically masked. Only used for visualizing, not for calculating.

{dendrogram,colors}_ratio : float, or pair of floats, optional

Proportion of the figure size devoted to the two marginal elements. If a pair is given, they correspond to (row, col) ratios.

cbar_pos : tuple of (left, bottom, width, height), optional

Position of the colorbar axes in the figure. Setting to ``None`` will disable the colorbar.

tree_kws : dict, optional

Parameters for the :class:`matplotlib.collections.LineCollection` that is used to plot the lines of the dendrogram tree.

kwargs : other keyword arguments

All other keyword arguments are passed to :func:`heatmap`.

Returns

:class:`ClusterGrid`

A :class:`ClusterGrid` instance.

See Also

heatmap : Plot rectangular data as a color-encoded matrix.

Notes

The returned object has a ``savefig`` method that should be used if you want to save the figure object without clipping the dendrograms.

To access the reordered row indices, use:

```
``clustergrid.dendrogram_row.reordered_ind``
```

Column indices, use:

```
``clustergrid.dendrogram_col.reordered_ind``
```

Examples

```
.. include:: ../docstrings/clustermap.rst
```

```
heatmap(  
    data,  
    *,  
    vmin=None,  
    vmax=None,  
    cmap=None,
```

```
center=None,

robust=False,

annot=None,

fmt='.2g',

annot_kws=None,

linewidths=0,

linecolor='white',

cbar=True,

cbar_kws=None,

cbar_ax=None,

square=False,

xticklabels='auto',

yticklabels='auto',

mask=None,

ax=None,

**kwargs

)
```

Plot rectangular data as a color-encoded matrix.

This is an Axes-level function and will draw the heatmap into the currently-active Axes if none is provided to the ``ax`` argument. Part of this Axes space will be taken and used to plot a colormap, unless ``cbar`` is False or a separate Axes is provided to ``cbar\_ax``.

Parameters

data : rectangular dataset

2D dataset that can be coerced into an ndarray. If a Pandas DataFrame is provided, the index/column information will be used to label the columns and rows.

`vmin, vmax` : floats, optional

Values to anchor the colormap, otherwise they are inferred from the data and other keyword arguments.

`cmap` : matplotlib colormap name or object, or list of colors, optional

The mapping from data values to color space. If not provided, the default will depend on whether `center` is set.

`center` : float, optional

The value at which to center the colormap when plotting divergent data.

Using this parameter will change the default `cmap` if none is specified.

`robust` : bool, optional

If True and `vmin` or `vmax` are absent, the colormap range is computed with robust quantiles instead of the extreme values.

`annot` : bool or rectangular dataset, optional

If True, write the data value in each cell. If an array-like with the same shape as `data`, then use this to annotate the heatmap instead of the data. Note that DataFrames will match on position, not index.

`fmt` : str, optional

String formatting code to use when adding annotations.

`annot_kws` : dict of key, value mappings, optional

Keyword arguments for `matplotlib.axes.Axes.text` when `annot` is True.

`linewidths` : float, optional

Width of the lines that will divide each cell.

linecolor : color, optional

Color of the lines that will divide each cell.

cbar : bool, optional

Whether to draw a colorbar.

cbar_kws : dict of key, value mappings, optional

Keyword arguments for :meth:`matplotlib.figure.Figure.colorbar`.

cbar_ax : matplotlib Axes, optional

Axes in which to draw the colorbar, otherwise take space from the main Axes.

square : bool, optional

If True, set the Axes aspect to "equal" so each cell will be square-shaped.

xticklabels, yticklabels : "auto", bool, list-like, or int, optional

If True, plot the column names of the dataframe. If False, don't plot the column names. If list-like, plot these alternate labels as the xticklabels. If an integer, use the column names but plot only every n label. If "auto", try to densely plot non-overlapping labels.

mask : bool array or DataFrame, optional

If passed, data will not be shown in cells where ``mask`` is True. Cells with missing values are automatically masked.

ax : matplotlib Axes, optional

Axes in which to draw the plot, otherwise use the currently-active Axes.

kwargs : other keyword arguments

All other keyword arguments are passed to :meth:`matplotlib.axes.Axes.pcolormesh`.

Returns

ax : matplotlib Axes

Axes object with the heatmap.

See Also

clustermap : Plot a matrix using hierarchical clustering to arrange the
rows and columns.

Examples

.. include:: ../docstrings/heatmap.rst

DATA

__all__ = ['heatmap', 'clustermap']

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\matrix.py

Help on module seaborn.miscplot in seaborn:

NAME

seaborn.miscplot

FUNCTIONS

```
dogplot(*_, **__)
```

Who's a good boy?

```
palplot(pal, size=1)
```

Plot the values in a color palette as a horizontal array.

Parameters

pal : sequence of matplotlib colors

colors, i.e. as returned by `seaborn.color_palette()`

size :

scaling factor for size of plot

DATA

```
__all__ = ['palplot', 'dogplot']
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\miscplot.py

Help on module `seaborn.palettes` in `seaborn`:

NAME

`seaborn.palettes`

FUNCTIONS

```
blend_palette(colors, n_colors=6, as_cmap=False, input='rgb')
```

Make a palette that blends between a list of colors.

Parameters

colors : sequence of colors in various formats interpreted by ``input``

hex code, html color name, or tuple in ``input`` space.

n_colors : int, optional

Number of colors in the palette.

as_cmap : bool, optional

If True, return a `:class:`matplotlib.colors.ListedColormap``.

Returns

palette

list of RGB tuples or `:class:`matplotlib.colors.ListedColormap``

Examples

```
.. include: ../docstrings/blend_palette.rst
```

```
color_palette(palette=None, n_colors=None, desat=None, as_cmap=False)
```

Return a list of colors or continuous colormap defining a palette.

Possible ``palette`` values include:

- Name of a seaborn palette (deep, muted, bright, pastel, dark, colorblind)
- Name of matplotlib colormap

- 'husl' or 'hls'
- 'ch:<cube helix arguments>'
- 'light:<color>', 'dark:<color>', 'blend:<color>,<color>',
- A sequence of colors in any format matplotlib accepts

Calling this function with `palette=None` will return the current matplotlib color cycle.

This function can also be used in a `with` statement to temporarily set the color cycle for a plot or set of plots.

See the :ref:`tutorial <palette\_tutorial>` for more information.

Parameters

`palette` : None, string, or sequence, optional

Name of palette or None to return current palette. If a sequence, input colors are used but possibly cycled and desaturated.

`n_colors` : int, optional

Number of colors in the palette. If `None`, the default will depend on how `palette` is specified. Named palettes default to 6 colors, but grabbing the current palette or passing in a list of colors will not change the number of colors unless this is specified. Asking for more colors than exist in the palette will cause it to cycle. Ignored when `as_cmap` is True.

`desat` : float, optional

Proportion to desaturate each color by.

`as_cmap : bool`

If True, return a `:class:`matplotlib.colors.ListedColormap``.

Returns

list of RGB tuples or `:class:`matplotlib.colors.ListedColormap``

See Also

`set_palette` : Set the default color cycle for all plots.

`set_color_codes` : Reassign color codes like ```"b"```, ```"g"```, etc. to
colors from one of the seaborn palettes.

Examples

```
.. include:: ../docstrings/color_palette.rst
```

`crayon_palette(colors)`

Make a palette with color names from Crayola crayons.

Colors are taken from here:

https://en.wikipedia.org/wiki/List_of_Crayola_crayon_colors

This is just a simple wrapper around the ``seaborn.crayons`` dictionary.

Parameters

colors : list of strings

List of keys in the `seaborn.crayons` dictionary.

Returns

palette

A list of colors as RGB tuples.

See Also

xkcd_palette : Make a palette with named colors from the XKCD color survey.

cubehelix_palette(

n_colors=6,

start=0,

rot=0.4,

gamma=1.0,

hue=0.8,

light=0.85,

dark=0.15,

reverse=False,

as_cmap=False

)

Make a sequential palette from the cubehelix system.

This produces a colormap with linearly-decreasing (or increasing)

brightness. That means that information will be preserved if printed to black and white or viewed by someone who is colorblind. "cubehelix" is also available as a matplotlib-based palette, but this function gives the user more control over the look of the palette and has a different set of defaults.

In addition to using this function, it is also possible to generate a cubehelix palette generally in seaborn using a string starting with ``ch:`` and containing other parameters (e.g. ``"ch:s=.25,r=-.5"``).

Parameters

`n_colors : int`

Number of colors in the palette.

`start : float, 0 <= start <= 3`

The hue value at the start of the helix.

`rot : float`

Rotations around the hue wheel over the range of the palette.

`gamma : float 0 <= gamma`

Nonlinearity to emphasize dark ($\gamma < 1$) or light ($\gamma > 1$) colors.

`hue : float, 0 <= hue <= 1`

Saturation of the colors.

`dark : float 0 <= dark <= 1`

Intensity of the darkest color in the palette.

`light : float 0 <= light <= 1`

Intensity of the lightest color in the palette.

`reverse : bool`

If True, the palette will go from dark to light.

`as_cmap : bool`

If True, return a `:class:`matplotlib.colors.ListedColormap``.

Returns

palette

list of RGB tuples or `:class:`matplotlib.colors.ListedColormap``

See Also

`choose_cubehelix_palette` : Launch an interactive widget to select cubehelix
palette parameters.

`dark_palette` : Create a sequential palette with dark low values.

`light_palette` : Create a sequential palette with bright low values.

References

Green, D. A. (2011). "A colour scheme for the display of astronomical
intensity images". Bulletin of the Astronomical Society of India, Vol. 39,
p. 289-295.

Examples

```
.. include:: ../docstrings/cubehelix_palette.rst
```

```
dark_palette(color, n_colors=6, reverse=False, as_cmap=False, input='rgb')
```

Make a sequential palette that blends from dark to ``color``.

This kind of palette is good for data that range between relatively uninteresting low values and interesting high values.

The ``color`` parameter can be specified in a number of ways, including all options for defining a color in matplotlib and several additional color spaces that are handled by seaborn. You can also use the database of named colors from the XKCD color survey.

If you are using the IPython notebook, you can also choose this palette interactively with the `:func:`choose_dark_palette`` function.

Parameters

`color` : base color for high values

hex, rgb-tuple, or html color name

`n_colors` : int, optional

number of colors in the palette

`reverse` : bool, optional

if True, reverse the direction of the blend

`as_cmap` : bool, optional

If True, return a `:class:`matplotlib.colors.ListedColormap``.

`input` : {'rgb', 'hls', 'husl', 'xkcd'}

Color space to interpret the input color. The first three options

apply to tuple inputs and the latter applies to string inputs.

Returns

palette

list of RGB tuples or :class:`matplotlib.colors.ListedColormap`

See Also

light_palette : Create a sequential palette with bright low values.

diverging_palette : Create a diverging palette with two colors.

Examples

```
.. include:: ../docstrings/dark_palette.rst
```

diverging_palette(

h_neg,

h_pos,

s=75,

l=50,

sep=1,

n=6,

center='light',

as_cmap=False

)

Make a diverging palette between two HUSL colors.

If you are using the IPython notebook, you can also choose this palette

interactively with the `:func:`choose_diverging_palette`` function.

Parameters

`h_neg, h_pos` : float in [0, 359]

Anchor hues for negative and positive extents of the map.

`s` : float in [0, 100], optional

Anchor saturation for both extents of the map.

`l` : float in [0, 100], optional

Anchor lightness for both extents of the map.

`sep` : int, optional

Size of the intermediate region.

`n` : int, optional

Number of colors in the palette (if not returning a cmap)

`center` : {"light", "dark"}, optional

Whether the center of the palette is light or dark

`as_cmap` : bool, optional

If True, return a `:class:`matplotlib.colors.ListedColormap``.

Returns

palette

list of RGB tuples or `:class:`matplotlib.colors.ListedColormap``

See Also

`dark_palette` : Create a sequential palette with dark values.

`light_palette` : Create a sequential palette with light values.

Examples

```
.. include: ../docstrings/diverging_palette.rst
```

```
hls_palette(n_colors=6, h=0.01, l=0.6, s=0.65, as_cmap=False)
```

Return hues with constant lightness and saturation in the HLS system.

The hues are evenly sampled along a circular path. The resulting palette will be appropriate for categorical or cyclical data.

The ``h``, ``l``, and ``s`` values should be between 0 and 1.

.. note::

While the separation of the resulting colors will be mathematically constant, the HLS system does not construct a perceptually-uniform space, so their apparent intensity will vary.

Parameters

`n_colors` : int

Number of colors in the palette.

`h` : float

The value of the first hue.

`l` : float

The lightness value.

`s : float`

The saturation intensity.

`as_cmap : bool`

If True, return a matplotlib colormap object.

Returns

palette

list of RGB tuples or :class:`matplotlib.colors.ListedColormap`

See Also

`hsl_palette` : Make a palette using evenly spaced hues in the HUSL system.

Examples

```
.. include:: ../docstrings/hls_palette.rst
```

```
hsl_palette(n_colors=6, h=0.01, s=0.9, l=0.65, as_cmap=False)
```

Return hues with constant lightness and saturation in the HUSL system.

The hues are evenly sampled along a circular path. The resulting palette will be appropriate for categorical or cyclical data.

The ``h``, ``l``, and ``s`` values should be between 0 and 1.

This function is similar to :func:`hls\_palette`, but it uses a nonlinear color

space that is more perceptually uniform.

Parameters

`n_colors : int`

Number of colors in the palette.

`h : float`

The value of the first hue.

`l : float`

The lightness value.

`s : float`

The saturation intensity.

`as_cmap : bool`

If True, return a matplotlib colormap object.

Returns

`palette`

list of RGB tuples or :class:`matplotlib.colors.ListedColormap`

See Also

`hls_palette` : Make a palette using evenly spaced hues in the HSL system.

Examples

```
.. include:: ../docstrings/husl_palette.rst
```

```
light_palette(color, n_colors=6, reverse=False, as_cmap=False, input='rgb')
```

Make a sequential palette that blends from light to ``color``.

The ``color`` parameter can be specified in a number of ways, including all options for defining a color in matplotlib and several additional color spaces that are handled by seaborn. You can also use the database of named colors from the XKCD color survey.

If you are using a Jupyter notebook, you can also choose this palette interactively with the :func:`choose\_light\_palette` function.

Parameters

color : base color for high values

hex code, html color name, or tuple in `input` space.

n_colors : int, optional

number of colors in the palette

reverse : bool, optional

if True, reverse the direction of the blend

as_cmap : bool, optional

If True, return a :class:`matplotlib.colors.ListedColormap`.

input : {'rgb', 'hls', 'husl', 'xkcd'}

Color space to interpret the input color. The first three options apply to tuple inputs and the latter applies to string inputs.

Returns

palette

list of RGB tuples or :class:`matplotlib.colors.ListedColormap`

See Also

dark_palette : Create a sequential palette with dark low values.

diverging_palette : Create a diverging palette with two colors.

Examples

```
.. include:: ../docstrings/light_palette.rst
```

`mpl_palette(name, n_colors=6, as_cmap=False)`

Return a palette or colormap from the matplotlib registry.

For continuous palettes, evenly-spaced discrete samples are chosen while excluding the minimum and maximum value in the colormap to provide better contrast at the extremes.

For qualitative palettes (e.g. those from colorbrewer), exact values are indexed (rather than interpolated), but fewer than ``n_colors`` can be returned if the palette does not define that many.

Parameters

name : string

Name of the palette. This should be a named matplotlib colormap.

`n_colors : int`

Number of discrete colors in the palette.

Returns

list of RGB tuples or :class:`matplotlib.colors.ListedColormap`

Examples

```
.. include:: ../docstrings/mpl_palette.rst
```

`set_color_codes(palette='deep')`

Change how matplotlib color shorthands are interpreted.

Calling this will change how shorthand codes like "b" or "g" are interpreted by matplotlib in subsequent plots.

Parameters

`palette : {deep, muted, pastel, dark, bright, colorblind}`

Named seaborn palette to use as the source of colors.

See Also

`set` : Color codes can be set through the high-level seaborn style manager.

`set_palette` : Color codes can also be set through the function that
sets the matplotlib color cycle.

`xkcd_palette(colors)`

Make a palette with color names from the xkcd color survey.

See xkcd for the full list of colors: <https://xkcd.com/color/rgb/>

This is just a simple wrapper around the ``seaborn.xkcd_rgb`` dictionary.

Parameters

`colors` : list of strings

List of keys in the ``seaborn.xkcd_rgb`` dictionary.

Returns

`palette`

A list of colors as RGB tuples.

See Also

`crayon_palette` : Make a palette with Crayola crayon colors.

DATA

`__all__` = ['color_palette', 'hls_palette', 'husl_palette', 'mpl_palett...

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\palettes.py

Help on module seaborn.rcmod in seaborn:

NAME

seaborn.rcmod - Control plot style and scaling using the matplotlib rcParams interface.

FUNCTIONS

`axes_style(style=None, rc=None)`

Get the parameters that control the general style of the plots.

The style parameters control properties like the color of the background and whether a grid is enabled by default. This is accomplished using the matplotlib rcParams system.

The options are illustrated in the

:doc:`aesthetics tutorial <../tutorial/aesthetics>`.

This function can also be used as a context manager to temporarily alter the global defaults. See :func:`set\_theme` or :func:`set\_style` to modify the global defaults for all plots.

Parameters

style : None, dict, or one of {darkgrid, whitegrid, dark, white, ticks}

A dictionary of parameters or the name of a preconfigured style.

`rc` : dict, optional

Parameter mappings to override the values in the preset seaborn style dictionaries. This only updates parameters that are considered part of the style definition.

Examples

```
.. include:: ../docstrings/axes_style.rst
```

```
plotting_context(context=None, font_scale=1, rc=None)
```

Get the parameters that control the scaling of plot elements.

These parameters correspond to label size, line thickness, etc. For more information, see the :doc:`aesthetics tutorial <../tutorial/aesthetics>`.

The base context is "notebook", and the other contexts are "paper", "talk", and "poster", which are version of the notebook parameters scaled by different values. Font elements can also be scaled independently of (but relative to) the other values.

This function can also be used as a context manager to temporarily alter the global defaults. See :func:`set\_theme` or :func:`set\_context` to modify the global defaults for all plots.

Parameters

context : None, dict, or one of {paper, notebook, talk, poster}

A dictionary of parameters or the name of a preconfigured set.

font_scale : float, optional

Separate scaling factor to independently scale the size of the font elements.

rc : dict, optional

Parameter mappings to override the values in the preset seaborn context dictionaries. This only updates parameters that are considered part of the context definition.

Examples

```
.. include:: ../docstrings/plotting_context.rst
```

reset_defaults()

Restore all RC params to default settings.

reset_orig()

Restore all RC params to original settings (respects custom rc).

set(*args, **kwargs)

Alias for :func:`set\_theme`, which is the preferred interface.

This function may be removed in the future.

```
set_context(context=None, font_scale=1, rc=None)
```

Set the parameters that control the scaling of plot elements.

These parameters correspond to label size, line thickness, etc.

Calling this function modifies the global matplotlib `rcParams`. For more information, see the :doc:`aesthetics tutorial <../tutorial/aesthetics>`.

The base context is "notebook", and the other contexts are "paper", "talk", and "poster", which are version of the notebook parameters scaled by different values. Font elements can also be scaled independently of (but relative to) the other values.

See :func:`plotting\_context` to get the parameter values.

Parameters

`context` : dict, or one of {paper, notebook, talk, poster}

A dictionary of parameters or the name of a preconfigured set.

`font_scale` : float, optional

Separate scaling factor to independently scale the size of the font elements.

`rc` : dict, optional

Parameter mappings to override the values in the preset seaborn context dictionaries. This only updates parameters that are considered part of the context definition.

Examples

`.. include:: ../docstrings/set_context.rst`

`set_palette(palette, n_colors=None, desat=None, color_codes=False)`

Set the matplotlib color cycle using a seaborn palette.

Parameters

`palette` : seaborn color palette | matplotlib colormap | hls | husl

Palette definition. Should be something `:func:`color_palette`` can process.

`n_colors` : int

Number of colors in the cycle. The default number of colors will depend on the format of ```palette```, see the `:func:`color_palette`` documentation for more information.

`desat` : float

Proportion to desaturate each color by.

`color_codes` : bool

If ```True``` and ```palette``` is a seaborn palette, remap the shorthand color codes (e.g. "b", "g", "r", etc.) to the colors from this palette.

See Also

`color_palette` : build a color palette or set the color cycle temporarily in a ```with``` statement.

`set_context` : set parameters to scale plot elements

`set_style` : set the default parameters for figure style

`set_style(style=None, rc=None)`

Set the parameters that control the general style of the plots.

The style parameters control properties like the color of the background and whether a grid is enabled by default. This is accomplished using the `matplotlib rcParams` system.

The options are illustrated in the

`:doc:`aesthetics tutorial <../tutorial/aesthetics>`.`

See `:func:`axes_style`` to get the parameter values.

Parameters

`style` : dict, or one of {darkgrid, whitegrid, dark, white, ticks}

A dictionary of parameters or the name of a preconfigured style.

`rc` : dict, optional

Parameter mappings to override the values in the preset seaborn style dictionaries. This only updates parameters that are considered part of the style definition.

Examples

`.. include:: ../docstrings/set_style.rst`

```

set_theme(

    context='notebook',

    style='darkgrid',

    palette='deep',

    font='sans-serif',

    font_scale=1,

    color_codes=True,

    rc=None

)

```

Set aspects of the visual theme for all matplotlib and seaborn plots.

This function changes the global defaults for all plots using the matplotlib rcParams system. The themeing is decomposed into several distinct sets of parameter values.

The options are illustrated in the :doc:`aesthetics <../tutorial/aesthetics>` and :doc:`color palette <../tutorial/color\_palettes>` tutorials.

Parameters

context : string or dict

Scaling parameters, see :func:`plotting\_context`.

style : string or dict

Axes style parameters, see :func:`axes\_style`.

palette : string or sequence

Color palette, see :func:`color\_palette`.

font : string

Font family, see matplotlib font manager.

font_scale : float, optional

Separate scaling factor to independently scale the size of the
font elements.

color_codes : bool

If ``True`` and ``palette`` is a seaborn palette, remap the shorthand
color codes (e.g. "b", "g", "r", etc.) to the colors from this palette.

rc : dict or None

Dictionary of rc parameter mappings to override the above.

Examples

```
.. include:: ../docstrings/set_theme.rst
```

DATA

```
__all__ = ['set_theme', 'set', 'reset_defaults', 'reset_orig', 'axes_s...
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\rcmod.py
```

Help on module seaborn.regression in seaborn:

NAME

seaborn.regression - Plotting functions for linear models (broadly construed).

FUNCTIONS

```
lmp1ot(  
  
    data,  
  
    *,  
  
    x=None,  
  
    y=None,  
  
    hue=None,  
  
    col=None,  
  
    row=None,  
  
    palette=None,  
  
    col_wrap=None,  
  
    height=5,  
  
    aspect=1,  
  
    markers='o',  
  
    sharex=None,  
  
    sharey=None,  
  
    hue_order=None,  
  
    col_order=None,  
  
    row_order=None,  
  
    legend=True,  
  
    legend_out=None,  
  
    x_estimator=None,  
  
    x_bins=None,  
  
    x_ci='ci',  
  
    scatter=True,  
  
    fit_reg=True,  
  
    ci=95,
```

```
n_boot=1000,  
  
units=None,  
  
seed=None,  
  
order=1,  
  
logistic=False,  
  
lowess=False,  
  
robust=False,  
  
logx=False,  
  
x_partial=None,  
  
y_partial=None,  
  
truncate=True,  
  
x_jitter=None,  
  
y_jitter=None,  
  
scatter_kws=None,  
  
line_kws=None,  
  
facet_kws=None  
)
```

Plot data and regression model fits across a FacetGrid.

This function combines :func:`regplot` and :class:`FacetGrid`. It is intended as a convenient interface to fit regression models across conditional subsets of a dataset.

When thinking about how to assign variables to different facets, a general rule is that it makes sense to use ``hue`` for the most important comparison, followed by ``col`` and ``row``. However, always think about your particular dataset and the goals of the visualization you are

creating.

There are a number of mutually exclusive options for estimating the regression model. See the :ref:`tutorial <regression\_tutorial>` for more information.

The parameters to this function span most of the options in :class:`FacetGrid`, although there may be occasional cases where you will want to use that class and :func:`regplot` directly.

Parameters

data : DataFrame

Tidy ("long-form") dataframe where each column is a variable and each row is an observation.

x, y : strings, optional

Input variables; these should be column names in ``data``.

hue, col, row : strings

Variables that define subsets of the data, which will be drawn on separate facets in the grid. See the ``\*\_order`` parameters to control the order of levels of this variable.

palette : palette name, list, or dict

Colors to use for the different levels of the ``hue`` variable. Should be something that can be interpreted by :func:`color\_palette`, or a dictionary mapping hue levels to matplotlib colors.

col_wrap : int

"Wrap" the column variable at this width, so that the column facets

span multiple rows. Incompatible with a ``row`` facet.

height : scalar

Height (in inches) of each facet. See also: ``aspect``.

aspect : scalar

Aspect ratio of each facet, so that ``aspect \* height`` gives the width of each facet in inches.

markers : matplotlib marker code or list of marker codes, optional

Markers for the scatterplot. If a list, each marker in the list will be used for each level of the ``hue`` variable.

share{x,y} : bool, 'col', or 'row' optional

If true, the facets will share y axes across columns and/or x axes across rows.

.. deprecated:: 0.12.0

Pass using the ``facet\_kws`` dictionary.

{hue,col,row}_order : lists, optional

Order for the levels of the faceting variables. By default, this will be the order that the levels appear in ``data`` or, if the variables are pandas categoricals, the category order.

legend : bool, optional

If ``True`` and there is a ``hue`` variable, add a legend.

legend_out : bool

If ``True``, the figure size will be extended, and the legend will be drawn outside the plot on the center right.

.. deprecated:: 0.12.0

Pass using the ``facet_kws`` dictionary.

`x_estimator` : callable that maps vector -> scalar, optional

Apply this function to each unique value of ``x`` and plot the resulting estimate. This is useful when ``x`` is a discrete variable. If ``x_ci`` is given, this estimate will be bootstrapped and a confidence interval will be drawn.

`x_bins` : int or vector, optional

Bin the ``x`` variable into discrete bins and then estimate the central tendency and a confidence interval. This binning only influences how the scatterplot is drawn; the regression is still fit to the original data. This parameter is interpreted either as the number of evenly-sized (not necessary spaced) bins or the positions of the bin centers. When this parameter is used, it implies that the default of ``x_estimator`` is ``numpy.mean``.

`x_ci` : "ci", "sd", int in [0, 100] or None, optional

Size of the confidence interval used when plotting a central tendency for discrete values of ``x``. If ``"ci"``, defer to the value of the ``ci`` parameter. If ``"sd"``, skip bootstrapping and show the standard deviation of the observations in each bin.

`scatter` : bool, optional

If ``True``, draw a scatterplot with the underlying observations (or the ``x_estimator`` values).

`fit_reg` : bool, optional

If ``True``, estimate and plot a regression model relating the ``x`` and ``y`` variables.

`ci` : int in [0, 100] or None, optional

Size of the confidence interval for the regression estimate. This will be drawn using translucent bands around the regression line. The confidence interval is estimated using a bootstrap; for large datasets, it may be advisable to avoid that computation by setting this parameter to None.

`n_boot` : int, optional

Number of bootstrap resamples used to estimate the `ci`. The default value attempts to balance time and stability; you may want to increase this value for "final" versions of plots.

`units` : variable name in `data`, optional

If the `x` and `y` observations are nested within sampling units, those can be specified here. This will be taken into account when computing the confidence intervals by performing a multilevel bootstrap that resamples both units and observations (within unit). This does not otherwise influence how the regression is estimated or drawn.

`seed` : int, `numpy.random.Generator`, or `numpy.random.RandomState`, optional

Seed or random number generator for reproducible bootstrapping.

`order` : int, optional

If `order` is greater than 1, use `numpy.polyfit` to estimate a polynomial regression.

`logistic` : bool, optional

If `True`, assume that `y` is a binary variable and use `statsmodels` to estimate a logistic regression model. Note that this is substantially more computationally intensive than linear regression, so you may wish to decrease the number of bootstrap resamples (`n_boot`) or set `ci` to None.

`lowess` : bool, optional

If `True`, use `statsmodels` to estimate a nonparametric lowess model (locally weighted linear regression). Note that confidence intervals cannot currently be drawn for this kind of model.

`robust` : bool, optional

If `True`, use `statsmodels` to estimate a robust regression. This will de-weight outliers. Note that this is substantially more computationally intensive than standard linear regression, so you may wish to decrease the number of bootstrap resamples (`n_boot`) or set `ci` to None.

`logx` : bool, optional

If `True`, estimate a linear regression of the form $y \sim \log(x)$, but plot the scatterplot and regression model in the input space. Note that `x` must be positive for this to work.

`{x,y}_partial` : strings in `data` or matrices

Confounding variables to regress out of the `x` or `y` variables before plotting.

`truncate` : bool, optional

If `True`, the regression line is bounded by the data limits. If `False`, it extends to the `x` axis limits.

`{x,y}_jitter` : floats, optional

Add uniform random noise of this size to either the `x` or `y` variables. The noise is added to a copy of the data after fitting the regression, and only influences the look of the scatterplot. This can be helpful when plotting variables that take discrete values.

`{scatter,line}_kws` : dictionaries

Additional keyword arguments to pass to `plt.scatter` and

```
`plt.plot`.
```

facet_kws : dict

Dictionary of keyword arguments for :class:`FacetGrid`.

See Also

regplot : Plot data and a conditional model fit.

FacetGrid : Subplot grid for plotting conditional relationships.

pairplot : Combine :func:`regplot` and :class:`PairGrid` (when used with
`kind="reg"`).

Notes

The :func:`regplot` and :func:`lmpplot` functions are closely related, but the former is an axes-level function while the latter is a figure-level function that combines :func:`regplot` and :class:`FacetGrid`.

Examples

```
.. include:: ../docstrings/lmplot.rst
```

regplot(

data=None,

*,

x=None,

```
y=None,  
  
x_estimator=None,  
  
x_bins=None,  
  
x_ci='ci',  
  
scatter=True,  
  
fit_reg=True,  
  
ci=95,  
  
n_boot=1000,  
  
units=None,  
  
seed=None,  
  
order=1,  
  
logistic=False,  
  
lowess=False,  
  
robust=False,  
  
logx=False,  
  
x_partial=None,  
  
y_partial=None,  
  
truncate=True,  
  
dropna=True,  
  
x_jitter=None,  
  
y_jitter=None,  
  
label=None,  
  
color=None,  
  
marker='o',  
  
scatter_kws=None,  
  
line_kws=None,  
  
ax=None
```

)

Plot data and a linear regression model fit.

There are a number of mutually exclusive options for estimating the regression model. See the :ref:`tutorial <regression\_tutorial>` for more information.

Parameters

`x, y`: string, series, or vector array

Input variables. If strings, these should correspond with column names in ``data``. When pandas objects are used, axes will be labeled with the series name.

`data` : DataFrame

Tidy ("long-form") dataframe where each column is a variable and each row is an observation.

`x_estimator` : callable that maps vector -> scalar, optional

Apply this function to each unique value of ``x`` and plot the resulting estimate. This is useful when ``x`` is a discrete variable.

If ``x\_ci`` is given, this estimate will be bootstrapped and a confidence interval will be drawn.

`x_bins` : int or vector, optional

Bin the ``x`` variable into discrete bins and then estimate the central tendency and a confidence interval. This binning only influences how the scatterplot is drawn; the regression is still fit to the original data. This parameter is interpreted either as the number of evenly-sized (not necessary spaced) bins or the positions of the bin

centers. When this parameter is used, it implies that the default of

```x_estimator``` is ```numpy.mean```.

`x_ci` : "ci", "sd", int in [0, 100] or None, optional

Size of the confidence interval used when plotting a central tendency for discrete values of ```x```. If ```"ci"```, defer to the value of the ```ci``` parameter. If ```"sd"```, skip bootstrapping and show the standard deviation of the observations in each bin.

`scatter` : bool, optional

If ```True```, draw a scatterplot with the underlying observations (or the ```x_estimator``` values).

`fit_reg` : bool, optional

If ```True```, estimate and plot a regression model relating the ```x``` and ```y``` variables.

`ci` : int in [0, 100] or None, optional

Size of the confidence interval for the regression estimate. This will be drawn using translucent bands around the regression line. The confidence interval is estimated using a bootstrap; for large datasets, it may be advisable to avoid that computation by setting this parameter to None.

`n_boot` : int, optional

Number of bootstrap resamples used to estimate the ```ci```. The default value attempts to balance time and stability; you may want to increase this value for "final" versions of plots.

`units` : variable name in ```data```, optional

If the ```x``` and ```y``` observations are nested within sampling units, those can be specified here. This will be taken into account when computing the confidence intervals by performing a multilevel bootstrap

that resamples both units and observations (within unit). This does not otherwise influence how the regression is estimated or drawn.

seed : int, numpy.random.Generator, or numpy.random.RandomState, optional

Seed or random number generator for reproducible bootstrapping.

order : int, optional

If ``order`` is greater than 1, use ``numpy.polyfit`` to estimate a polynomial regression.

logistic : bool, optional

If ``True``, assume that ``y`` is a binary variable and use ``statsmodels`` to estimate a logistic regression model. Note that this is substantially more computationally intensive than linear regression, so you may wish to decrease the number of bootstrap resamples (``n\_boot``) or set ``ci`` to None.

lowess : bool, optional

If ``True``, use ``statsmodels`` to estimate a nonparametric lowess model (locally weighted linear regression). Note that confidence intervals cannot currently be drawn for this kind of model.

robust : bool, optional

If ``True``, use ``statsmodels`` to estimate a robust regression. This will de-weight outliers. Note that this is substantially more computationally intensive than standard linear regression, so you may wish to decrease the number of bootstrap resamples (``n\_boot``) or set ``ci`` to None.

logx : bool, optional

If ``True``, estimate a linear regression of the form  $y \sim \log(x)$ , but plot the scatterplot and regression model in the input space. Note that ``x`` must be positive for this to work.

`{x,y}_partial` : strings in ``data`` or matrices

Confounding variables to regress out of the ``x`` or ``y`` variables before plotting.

`truncate` : bool, optional

If ``True``, the regression line is bounded by the data limits. If ``False``, it extends to the ``x`` axis limits.

`{x,y}_jitter` : floats, optional

Add uniform random noise of this size to either the ``x`` or ``y`` variables. The noise is added to a copy of the data after fitting the regression, and only influences the look of the scatterplot. This can be helpful when plotting variables that take discrete values.

`label` : string

Label to apply to either the scatterplot or regression line (if ``scatter`` is ``False``) for use in a legend.

`color` : matplotlib color

Color to apply to all plot elements; will be superseded by colors passed in ``scatter\_kws`` or ``line\_kws``.

`marker` : matplotlib marker code

Marker to use for the scatterplot glyphs.

`{scatter,line}_kws` : dictionaries

Additional keyword arguments to pass to ``plt.scatter`` and ``plt.plot``.

`ax` : matplotlib Axes, optional

Axes object to draw the plot onto, otherwise uses the current Axes.

Returns

-----

`ax` : matplotlib Axes

The Axes object containing the plot.

See Also

-----

`lmpplot` : Combine `:func:`regplot`` and `:class:`FacetGrid`` to plot multiple linear relationships in a dataset.

`jointplot` : Combine `:func:`regplot`` and `:class:`JointGrid`` (when used with ```kind="reg"```).

`pairplot` : Combine `:func:`regplot`` and `:class:`PairGrid`` (when used with ```kind="reg"```).

`residplot` : Plot the residuals of a linear regression model.

Notes

-----

The `:func:`regplot`` and `:func:`lmpplot`` functions are closely related, but the former is an axes-level function while the latter is a figure-level function that combines `:func:`regplot`` and `:class:`FacetGrid``.

It's also easy to combine `:func:`regplot`` and `:class:`JointGrid`` or `:class:`PairGrid`` through the `:func:`jointplot`` and `:func:`pairplot`` functions, although these do not directly accept all of `:func:`regplot``'s parameters.

## Examples

-----

```
.. include:: ../docstrings/regplot.rst
```

```
residplot(
 data=None,
 *,
 x=None,
 y=None,
 x_partial=None,
 y_partial=None,
 lowess=False,
 order=1,
 robust=False,
 dropna=True,
 label=None,
 color=None,
 scatter_kws=None,
 line_kws=None,
 ax=None
)
```

Plot the residuals of a linear regression.

This function will regress  $y$  on  $x$  (possibly as a robust or polynomial regression) and then draw a scatterplot of the residuals. You can optionally fit a lowess smoother to the residual plot, which can

help in determining if there is structure to the residuals.

#### Parameters

-----

`data` : DataFrame, optional

DataFrame to use if ``x`` and ``y`` are column names.

`x` : vector or string

Data or column name in ``data`` for the predictor variable.

`y` : vector or string

Data or column name in ``data`` for the response variable.

`{x, y}_partial` : vectors or string(s) , optional

These variables are treated as confounding and are removed from the ``x`` or ``y`` variables before plotting.

`lowess` : boolean, optional

Fit a lowess smoother to the residual scatterplot.

`order` : int, optional

Order of the polynomial to fit when calculating the residuals.

`robust` : boolean, optional

Fit a robust linear regression when calculating the residuals.

`dropna` : boolean, optional

If True, ignore observations with missing data when fitting and plotting.

`label` : string, optional

Label that will be used in any plot legends.

`color` : matplotlib color, optional

Color to use for all elements of the plot.

`{scatter, line}_kws` : dictionaries, optional

Additional keyword arguments passed to `scatter()` and `plot()` for drawing the components of the plot.

`ax` : matplotlib axis, optional

Plot into this axis, otherwise grab the current axis or make a new one if not existing.

#### Returns

-----

`ax`: matplotlib axes

Axes with the regression plot.

#### See Also

-----

`regplot` : Plot a simple linear regression model.

`jointplot` : Draw a `:func:`residplot`` with univariate marginal distributions (when used with ```kind="resid"```).

#### Examples

-----

```
.. include:: ../docstrings/residplot.rst
```

#### DATA

```
__all__ = ['lmpplot', 'regplot', 'residplot']
```

#### FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\regression.py

Help on module seaborn.relational in seaborn:

## NAME

seaborn.relational

## FUNCTIONS

```
lineplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 size=None,
 style=None,
 units=None,
 weights=None,
 palette=None,
 hue_order=None,
 hue_norm=None,
 sizes=None,
 size_order=None,
 size_norm=None,
 dashes=True,
 markers=None,
 style_order=None,
```



```
estimator='mean',

errorbar=('ci', 95),

n_boot=1000,

seed=None,

orient='x',

sort=True,

err_style='band',

err_kws=None,

legend='auto',

ci='deprecated',

ax=None,

**kwargs
)
```

Draw a line plot with possibility of several semantic groupings.

The relationship between ``x`` and ``y`` can be shown for different subsets of the data using the ``hue``, ``size``, and ``style`` parameters. These parameters control what visual semantics are used to identify the different subsets. It is possible to show up to three dimensions independently by using all three semantic types, but this style of plot can be hard to interpret and is often ineffective. Using redundant semantics (i.e. both ``hue`` and ``style`` for the same variable) can be helpful for making graphics more accessible.

See the :ref:`tutorial <relational\_tutorial>` for more information.

The default treatment of the ``hue`` (and to a lesser extent, ``size``)

semantic, if present, depends on whether the variable is inferred to represent "numeric" or "categorical" data. In particular, numeric variables are represented with a sequential colormap by default, and the legend entries show regular "ticks" with values that may or may not exist in the data. This behavior can be controlled through various parameters, as described and illustrated below.

By default, the plot aggregates over multiple `y`` values at each value of `x`` and shows an estimate of the central tendency and a confidence interval for that estimate.

#### Parameters

-----

`data` : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

`x, y` : vectors or keys in ```data```

Variables that specify positions on the x and y axes.

`hue` : vector or key in ``data``

Grouping variable that will produce lines with different colors.

Can be either categorical or numeric, although color mapping will behave differently in latter case.

`size` : vector or key in ``data``

Grouping variable that will produce lines with different widths.

Can be either categorical or numeric, although size mapping will behave differently in latter case.

`style` : vector or key in ``data``

Grouping variable that will produce lines with different dashes and/or markers. Can have a numeric dtype but will always be treated as categorical.

`units` : vector or key in ``data``

Grouping variable identifying sampling units. When used, a separate line will be drawn for each unit with appropriate semantics, but no legend entry will be added. Useful for showing distribution of experimental replicates when exact identities are not needed.

`weights` : vector or key in ``data``

Data values or column used to compute weighted estimation.

Note that use of weights currently limits the choice of statistics to a 'mean' estimator and 'ci' errorbar.

`palette` : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ```hue``` semantic.

String values are passed to :func:`color\_palette`. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order` : vector of strings

Specify the order of processing and plotting for categorical levels of the ```hue``` semantic.

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units or an object that will map from data units into a [0, 1] interval. Usage implies numeric mapping.

`sizes` : list, dict, or tuple

An object that determines how sizes are chosen when ``size`` is used.

List or dict arguments should provide a size for each unique data value,

which forces a categorical interpretation. The argument may also be a min, max tuple.

size\_order : list

Specified order for appearance of the `size` variable levels, otherwise they are determined from the data. Not relevant when the `size` variable is numeric.

size\_norm : tuple or Normalize object

Normalization in data units for scaling plot objects when the `size` variable is numeric.

dashes : boolean, list, or dictionary

Object determining how to draw the lines for different levels of the `style` variable. Setting to `True` will use default dash codes, or you can pass a list of dash codes or a dictionary mapping levels of the `style` variable to dash codes. Setting to `False` will use solid lines for all subsets. Dashes are specified as in matplotlib: a tuple of `(segment, gap)` lengths, or an empty string to draw a solid line.

markers : boolean, list, or dictionary

Object determining how to draw the markers for different levels of the `style` variable. Setting to `True` will use default markers, or you can pass a list of markers or a dictionary mapping levels of the `style` variable to markers. Setting to `False` will draw marker-less lines. Markers are specified as in matplotlib.

style\_order : list

Specified order for appearance of the `style` variable levels otherwise they are determined from the data. Not relevant when the `style` variable is numeric.

estimator : name of pandas method or callable or None

Method for aggregating across multiple observations of the `y`` variable at the same `x`` level. If `None``, all observations will be drawn.

`errorbar` : string, (string, number) tuple, or callable

Name of errorbar method (either "ci", "pi", "se", or "sd"), or a tuple with a method name and a level parameter, or a function that maps from a vector to a (min, max) interval, or None to hide errorbar. See the :doc:`errorbar tutorial </tutorial/errorBars>` for more information.

`n_boot` : int

Number of bootstraps to use for computing the confidence interval.

`seed` : int, `numpy.random.Generator`, or `numpy.random.RandomState`

Seed or random number generator for reproducible bootstrapping.

`orient` : "x" or "y"

Dimension along which the data are sorted / aggregated. Equivalently, the "independent variable" of the resulting function.

`sort` : boolean

If True, the data will be sorted by the x and y variables, otherwise lines will connect points in the order they appear in the dataset.

`err_style` : "band" or "bars"

Whether to draw the confidence intervals with translucent error bands or discrete error bars.

`err_kws` : dict of keyword arguments

Additional parameters to control the aesthetics of the error bars. The kwargs are passed either to :meth:`matplotlib.axes.Axes.fill\_between` or :meth:`matplotlib.axes.Axes.errorbar`, depending on `err_style``.

`legend` : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric `hue`` and `size``

variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If `False`, no legend data is added and no legend is drawn.

`ci : int or "sd" or None`

Size of the confidence interval to draw when aggregating.

.. deprecated:: 0.12.0

Use the new `errorbar` parameter for more flexibility.

`ax : :class:`matplotlib.axes.Axes``

Pre-existing axes for the plot. Otherwise, call `:func:`matplotlib.pyplot.gca`` internally.

`kwargs : key, value mappings`

Other keyword arguments are passed down to

`:meth:`matplotlib.axes.Axes.plot`.`

Returns

-----

`:class:`matplotlib.axes.Axes``

The matplotlib axes containing the plot.

See Also

-----

`scatterplot` : Plot data using points.

`pointplot` : Plot point estimates and CIs using markers and lines.

## Examples

-----

```
.. include:: ../docstrings/lineplot.rst
```

```
relplot(

 data=None,

 *,

 x=None,

 y=None,

 hue=None,

 size=None,

 style=None,

 units=None,

 weights=None,

 row=None,

 col=None,

 col_wrap=None,

 row_order=None,

 col_order=None,

 palette=None,

 hue_order=None,

 hue_norm=None,

 sizes=None,

 size_order=None,

 size_norm=None,

 markers=None,
```

```
dashes=None,

style_order=None,

legend='auto',

kind='scatter',

height=5,

aspect=1,

facet_kws=None,

**kwargs
)
```

Figure-level interface for drawing relational plots onto a FacetGrid.

This function provides access to several different axes-level functions that show the relationship between two variables with semantic mappings of subsets. The `kind` parameter selects the underlying axes-level function to use:

- :func:`scatterplot` (with `kind="scatter"`; the default)
- :func:`lineplot` (with `kind="line"`)

Extra keyword arguments are passed to the underlying function, so you should refer to the documentation for each to see kind-specific options.

The relationship between `x` and `y` can be shown for different subsets of the data using the `hue`, `size`, and `style` parameters. These parameters control what visual semantics are used to identify the different subsets. It is possible to show up to three dimensions independently by using all three semantic types, but this style of plot can be hard to



interpret and is often ineffective. Using redundant semantics (i.e. both ``hue`` and ``style`` for the same variable) can be helpful for making graphics more accessible.

See the :ref:`tutorial <relational\_tutorial>` for more information.

The default treatment of the ``hue`` (and to a lesser extent, ``size``) semantic, if present, depends on whether the variable is inferred to represent "numeric" or "categorical" data. In particular, numeric variables are represented with a sequential colormap by default, and the legend entries show regular "ticks" with values that may or may not exist in the data. This behavior can be controlled through various parameters, as described and illustrated below.

After plotting, the :class:`FacetGrid` with the plot is returned and can be used directly to tweak supporting plot details or add other layers.

## Parameters

-----

`data` : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

`x, y` : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

`hue` : vector or key in `data`

Grouping variable that will produce elements with different colors.

Can be either categorical or numeric, although color mapping will behave differently in latter case.

size : vector or key in `data`

Grouping variable that will produce elements with different sizes.

Can be either categorical or numeric, although size mapping will behave differently in latter case.

style : vector or key in `data`

Grouping variable that will produce elements with different styles.

Can have a numeric dtype but will always be treated as categorical.

units : vector or key in `data`

Grouping variable identifying sampling units. When used, a separate line will be drawn for each unit with appropriate semantics, but no legend entry will be added. Useful for showing distribution of experimental replicates when exact identities are not needed.

weights : vector or key in `data`

Data values or column used to compute weighted estimation.

Note that use of weights currently limits the choice of statistics to a 'mean' estimator and 'ci' errorbar.

row, col : vectors or keys in ``data``

Variables that define subsets to plot on different facets.

col\_wrap : int

"Wrap" the column variable at this width, so that the column facets span multiple rows. Incompatible with a ``row`` facet.

row\_order, col\_order : lists of strings

Order to organize the rows and/or columns of the grid in, otherwise the orders are inferred from the data objects.

palette : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to :func:`color\_palette`. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

hue\_order : vector of strings

Specify the order of processing and plotting for categorical levels of the ``hue`` semantic.

hue\_norm : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units or an object that will map from data units into a [0, 1] interval. Usage implies numeric mapping.

sizes : list, dict, or tuple

An object that determines how sizes are chosen when `size` is used.

List or dict arguments should provide a size for each unique data value, which forces a categorical interpretation. The argument may also be a min, max tuple.

size\_order : list

Specified order for appearance of the `size` variable levels, otherwise they are determined from the data. Not relevant when the `size` variable is numeric.

size\_norm : tuple or Normalize object

Normalization in data units for scaling plot objects when the `size` variable is numeric.

style\_order : list

Specified order for appearance of the `style` variable levels otherwise they are determined from the data. Not relevant when the `style` variable is numeric.

dashes : boolean, list, or dictionary

Object determining how to draw the lines for different levels of the ``style`` variable. Setting to ``True`` will use default dash codes, or you can pass a list of dash codes or a dictionary mapping levels of the ``style`` variable to dash codes. Setting to ``False`` will use solid lines for all subsets. Dashes are specified as in matplotlib: a tuple of ``(segment, gap)`` lengths, or an empty string to draw a solid line.

`markers` : boolean, list, or dictionary

Object determining how to draw the markers for different levels of the ``style`` variable. Setting to ``True`` will use default markers, or you can pass a list of markers or a dictionary mapping levels of the ``style`` variable to markers. Setting to ``False`` will draw marker-less lines. Markers are specified as in matplotlib.

`legend` : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size`` variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto", choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

`kind` : string

Kind of plot to draw, corresponding to a seaborn relational plot.

Options are ``"scatter"`` or ``"line"``.

`height` : scalar

Height (in inches) of each facet. See also: ``aspect``.

`aspect` : scalar

Aspect ratio of each facet, so that ``aspect * height`` gives the width of each facet in inches.

`facet_kws` : dict

Dictionary of other keyword arguments to pass to `:class:`FacetGrid``.

`kwargs` : key, value pairings

Other keyword arguments are passed through to the underlying plotting function.

Returns

-----

`:class:`FacetGrid``

An object managing one or more subplots that correspond to conditional data subsets with convenient methods for batch-setting of axes attributes.

Examples

-----

```
.. include:: ../docstrings/relplot.rst
```

```
scatterplot(
 data=None,
 *,
 x=None,
 y=None,
 hue=None,
 size=None,
 style=None,
 palette=None,
 hue_order=None,
 hue_norm=None,
```

```
sizes=None,

size_order=None,

size_norm=None,

markers=True,

style_order=None,

legend='auto',

ax=None,

**kwargs
)
```

Draw a scatter plot with possibility of several semantic groupings.

The relationship between ``x`` and ``y`` can be shown for different subsets of the data using the ``hue``, ``size``, and ``style`` parameters. These parameters control what visual semantics are used to identify the different subsets. It is possible to show up to three dimensions independently by using all three semantic types, but this style of plot can be hard to interpret and is often ineffective. Using redundant semantics (i.e. both ``hue`` and ``style`` for the same variable) can be helpful for making graphics more accessible.

See the :ref:`tutorial <relational\_tutorial>` for more information.

The default treatment of the ``hue`` (and to a lesser extent, ``size``) semantic, if present, depends on whether the variable is inferred to represent "numeric" or "categorical" data. In particular, numeric variables are represented with a sequential colormap by default, and the legend entries show regular "ticks" with values that may or may not exist in the

data. This behavior can be controlled through various parameters, as described and illustrated below.

#### Parameters

-----

`data` : :class:`pandas.DataFrame`, :class:`numpy.ndarray`, mapping, or sequence

Input data structure. Either a long-form collection of vectors that can be assigned to named variables or a wide-form dataset that will be internally reshaped.

`x, y` : vectors or keys in ``data``

Variables that specify positions on the x and y axes.

`hue` : vector or key in `data`

Grouping variable that will produce points with different colors.

Can be either categorical or numeric, although color mapping will behave differently in latter case.

`size` : vector or key in `data`

Grouping variable that will produce points with different sizes.

Can be either categorical or numeric, although size mapping will behave differently in latter case.

`style` : vector or key in `data`

Grouping variable that will produce points with different markers.

Can have a numeric dtype but will always be treated as categorical.

`palette` : string, list, dict, or :class:`matplotlib.colors.Colormap`

Method for choosing the colors to use when mapping the ``hue`` semantic.

String values are passed to :func:`color\_palette`. List or dict values imply categorical mapping, while a colormap object implies numeric mapping.

`hue_order` : vector of strings

Specify the order of processing and plotting for categorical levels of the

```hue``` semantic.

`hue_norm` : tuple or :class:`matplotlib.colors.Normalize`

Either a pair of values that set the normalization range in data units
or an object that will map from data units into a [0, 1] interval. Usage
implies numeric mapping.

`sizes` : list, dict, or tuple

An object that determines how sizes are chosen when `size`` is used.

List or dict arguments should provide a size for each unique data value,
which forces a categorical interpretation. The argument may also be a
min, max tuple.

`size_order` : list

Specified order for appearance of the `size`` variable levels,
otherwise they are determined from the data. Not relevant when the
`size`` variable is numeric.

`size_norm` : tuple or Normalize object

Normalization in data units for scaling plot objects when the
`size`` variable is numeric.

`markers` : boolean, list, or dictionary

Object determining how to draw the markers for different levels of the
`style`` variable. Setting to `True`` will use default markers, or
you can pass a list of markers or a dictionary mapping levels of the
`style`` variable to markers. Setting to `False`` will draw
marker-less lines. Markers are specified as in matplotlib.

`style_order` : list

Specified order for appearance of the `style`` variable levels
otherwise they are determined from the data. Not relevant when the

``style`` variable is numeric.

legend : "auto", "brief", "full", or False

How to draw the legend. If "brief", numeric ``hue`` and ``size``

variables will be represented with a sample of evenly spaced values.

If "full", every group will get an entry in the legend. If "auto",

choose between brief or full representation based on number of levels.

If ``False``, no legend data is added and no legend is drawn.

ax : :class:`matplotlib.axes.Axes`

Pre-existing axes for the plot. Otherwise, call :func:`matplotlib.pyplot.gca` internally.

kwargs : key, value mappings

Other keyword arguments are passed down to

:meth:`matplotlib.axes.Axes.scatter`.

Returns

:class:`matplotlib.axes.Axes`

The matplotlib axes containing the plot.

See Also

lineplot : Plot data using lines.

stripplot : Plot a categorical scatter with jitter.

swarmplot : Plot a categorical scatter with non-overlapping points.

Examples

```
.. include:: ../docstrings/scatterplot.rst
```

DATA

```
__all__ = ['relplot', 'scatterplot', 'lineplot']
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\relational.py
```

Help on module seaborn.utils in seaborn:

NAME

seaborn.utils - Utility functions, mostly for internal use.

FUNCTIONS

desaturate(color, prop)

Decrease the saturation channel of a color by some percent.

Parameters

color : matplotlib color

hex, rgb-tuple, or html color name

prop : float

saturation channel of color will be multiplied by this value

Returns

new_color : rgb tuple

desaturated color code in RGB tuple representation

despine(

fig=None,

ax=None,

top=True,

right=True,

left=False,

bottom=False,

offset=None,

trim=False

)

Remove the top and right spines from plot(s).

fig : matplotlib figure, optional

Figure to despine all axes of, defaults to the current figure.

ax : matplotlib axes, optional

Specific axes object to despine. Ignored if fig is provided.

top, right, left, bottom : boolean, optional

If True, remove that spine.

offset : int or dict, optional

Absolute distance, in points, spines should be moved away

from the axes (negative values move spines inward). A single value

applies to all spines; a dict can be used to set offset values per

side.

`trim : bool, optional`

If True, limit spines to the smallest and largest major tick
on each non-despined axis.

Returns

None

`get_data_home(data_home=None)`

Return a path to the cache directory for example datasets.

This directory is used by `:func:`load_dataset``.

If the ```data_home``` argument is not provided, it will use a directory
specified by the `SEABORN_DATA`` environment variable (if it exists)
or otherwise default to an OS-appropriate user cache location.

`get_dataset_names()`

Report available example datasets, useful for reporting issues.

Requires an internet connection.

`load_dataset(name, cache=True, data_home=None, **kws)`

Load an example dataset from the online repository (requires internet).

This function provides quick access to a small number of example datasets
that are useful for documenting seaborn or generating reproducible examples

for bug reports. It is not necessary for normal usage.

Note that some of the datasets have a small amount of preprocessing applied to define a proper ordering for categorical variables.

Use `:func:`get_dataset_names`` to see a list of available datasets.

Parameters

`name` : str

Name of the dataset (`{name}.csv`` on <https://github.com/mwaskom/seaborn-data>).

`cache` : boolean, optional

If True, try to load from the local cache first, and save to the cache if a download is required.

`data_home` : string, optional

The directory in which to cache data; see `:func:`get_data_home``.

`kws` : keys and values, optional

Additional keyword arguments are passed to passed through to `:func:`pandas.read_csv``.

Returns

`df` : `:class:`pandas.DataFrame``

Tabular data, possibly with some preprocessing applied.

`move_legend(obj, loc, **kwargs)`

Recreate a plot's legend at a new location.

The name is a slight misnomer. Matplotlib legends do not expose public control over their position parameters. So this function creates a new legend, copying over the data from the original object, which is then removed.

Parameters

`obj` : the object with the plot

This argument can be either a seaborn or matplotlib object:

- `:class:`seaborn.FacetGrid`` or `:class:`seaborn.PairGrid``
- `:class:`matplotlib.axes.Axes`` or `:class:`matplotlib.figure.Figure``

`loc` : str or int

Location argument, as in `:meth:`matplotlib.axes.Axes.legend``.

kwargs

Other keyword arguments are passed to `:meth:`matplotlib.axes.Axes.legend``.

Examples

```
.. include:: ../docstrings/move_legend.rst
```

`saturate(color)`

Return a fully saturated color with the same hue.

Parameters

color : matplotlib color

hex, rgb-tuple, or html color name

Returns

new_color : rgb tuple

saturated color code in RGB tuple representation

set_hls_values(color, h=None, l=None, s=None)

Independently manipulate the h, l, or s channels of a color.

Parameters

color : matplotlib color

hex, rgb-tuple, or html color name

h, l, s : floats between 0 and 1, or None

new values for each channel in hls space

Returns

new_color : rgb tuple

new color code in RGB tuple representation

```
__all__ = ['desaturate', 'saturate', 'set_hls_values', 'move_legend', ...
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\utils.py
```

Help on module seaborn.widgets in seaborn:

NAME

```
seaborn.widgets
```

FUNCTIONS

```
choose_colorbrewer_palette(data_type, as_cmap=False)
```

Select a palette from the ColorBrewer set.

These palettes are built into matplotlib and can be used by name in many seaborn functions, or by passing the object returned by this function.

Parameters

`data_type` : {'sequential', 'diverging', 'qualitative'}

This describes the kind of data you want to visualize. See the seaborn color palette docs for more information about how to choose this value.

Note that you can pass substrings (e.g. 'q' for 'qualitative).

`as_cmap` : bool

If True, the return value is a matplotlib colormap rather than a

list of discrete colors.

Returns

pal or cmap : list of colors or matplotlib colormap

Object that can be passed to plotting functions.

See Also

dark_palette : Create a sequential palette with dark low values.

light_palette : Create a sequential palette with bright low values.

diverging_palette : Create a diverging palette from selected colors.

cubehelix_palette : Create a sequential palette or colormap using the
cubehelix system.

choose_cubehelix_palette(as_cmap=False)

Launch an interactive widget to create a sequential cubehelix palette.

This corresponds with the :func:`cubehelix\_palette` function. This kind of palette is good for data that range between relatively uninteresting low values and interesting high values. The cubehelix system allows the palette to have more hue variance across the range, which can be helpful for distinguishing a wider range of values.

Requires IPython 2+ and must be used in the notebook.

Parameters

as_cmap : bool

If True, the return value is a matplotlib colormap rather than a list of discrete colors.

Returns

pal or cmap : list of colors or matplotlib colormap

Object that can be passed to plotting functions.

See Also

cubehelix_palette : Create a sequential palette or colormap using the cubehelix system.

choose_dark_palette(input='husl', as_cmap=False)

Launch an interactive widget to create a dark sequential palette.

This corresponds with the :func:`dark\_palette` function. This kind of palette is good for data that range between relatively uninteresting low values and interesting high values.

Requires IPython 2+ and must be used in the notebook.

Parameters

input : {'husl', 'hls', 'rgb'}

Color space for defining the seed value. Note that the default is different than the default input for `:func:`dark_palette``.

`as_cmap` : bool

If True, the return value is a matplotlib colormap rather than a list of discrete colors.

Returns

`pal` or `cmap` : list of colors or matplotlib colormap

Object that can be passed to plotting functions.

See Also

`dark_palette` : Create a sequential palette with dark low values.

`light_palette` : Create a sequential palette with bright low values.

`cubehelix_palette` : Create a sequential palette or colormap using the cubehelix system.

`choose_diverging_palette(as_cmap=False)`

Launch an interactive widget to choose a diverging color palette.

This corresponds with the `:func:`diverging_palette`` function. This kind of palette is good for data that range between interesting low values and interesting high values with a meaningful midpoint. (For example, change scores relative to some baseline value).

Requires IPython 2+ and must be used in the notebook.

Parameters

`as_cmap` : bool

If True, the return value is a matplotlib colormap rather than a list of discrete colors.

Returns

`pal` or `cmap` : list of colors or matplotlib colormap

Object that can be passed to plotting functions.

See Also

`diverging_palette` : Create a diverging color palette or colormap.

`choose_colorbrewer_palette` : Interactively choose palettes from the colorbrewer set, including diverging palettes.

`choose_light_palette(input='husl', as_cmap=False)`

Launch an interactive widget to create a light sequential palette.

This corresponds with the `:func:`light_palette`` function. This kind of palette is good for data that range between relatively uninteresting low values and interesting high values.

Requires IPython 2+ and must be used in the notebook.

Parameters

`input` : {'husl', 'hls', 'rgb'}

Color space for defining the seed value. Note that the default is different than the default input for `:func:`light_palette``.

`as_cmap` : bool

If True, the return value is a matplotlib colormap rather than a list of discrete colors.

Returns

`pal` or `cmap` : list of colors or matplotlib colormap

Object that can be passed to plotting functions.

See Also

`light_palette` : Create a sequential palette with bright low values.

`dark_palette` : Create a sequential palette with dark low values.

`cubehelix_palette` : Create a sequential palette or colormap using the cubehelix system.

DATA

```
__all__ = ['choose_colorbrewer_palette', 'choose_cubehelix_palette', '...
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\widgets.py

Help on module seaborn._base in seaborn:

NAME

seaborn._base

CLASSES

builtins.object

SemanticMapping

HueMapping

SizeMapping

StyleMapping

VectorPlotter

collections.UserString(collections.abc.Sequence)

VariableType

class HueMapping(SemanticMapping)

| HueMapping(plotter, palette=None, order=None, norm=None, saturation=1)

|

| Mapping that sets artist colors according to data values.

|

| Method resolution order:

| HueMapping

| SemanticMapping

| builtins.object

|

| Methods defined here:

```

|
|
|  __init__(self, plotter, palette=None, order=None, norm=None, saturation=1)
|
|      Map the levels of the `hue` variable to distinct colors.
|
|
|      Parameters
|
|      -----
|
|      # TODO add generic parameters
|
|
|  categorical_mapping(self, data, palette, order)
|
|      Determine colors when the hue mapping is categorical.
|
|
|  infer_map_type(self, palette, norm, input_format, var_type)
|
|      Determine how to implement the mapping.
|
|
|  numeric_mapping(self, data, palette, norm)
|
|      Determine colors when the hue variable is quantitative.
|
|
|  -----
|
|  Data and other attributes defined here:
|
|
|  cmap = None
|
|
|  norm = None
|
|
|  palette = None
|
|
|  -----

```

```

| Methods inherited from SemanticMapping:
|
|
| __call__(self, key, *args, **kwargs)
|
|     Get the attribute(s) values for the data key.
|
|
| -----
|
| Data descriptors inherited from SemanticMapping:
|
|
| __dict__
|
|     dictionary for instance variables
|
|
| __weakref__
|
|     list of weak references to the object
|
| -----
|
| Data and other attributes inherited from SemanticMapping:
|
|
| __annotations__ = {}
|
|
| levels = None
|
|
| lookup_table = None
|
|
| map_type = None
|
|
class SemanticMapping(builtins.object)
|
| SemanticMapping(plotter)

```



```
|  
  
| Base class for mapping data values to plot attributes.  
  
|  
  
| Methods defined here:  
  
|  
| __call__(self, key, *args, **kwargs)  
  
|     Get the attribute(s) values for the data key.  
  
|  
| __init__(self, plotter)  
  
|     Initialize self. See help(type(self)) for accurate signature.  
  
|  
| -----  
  
| Data descriptors defined here:  
  
|  
| __dict__  
  
|     dictionary for instance variables  
  
|  
| __weakref__  
  
|     list of weak references to the object  
  
|  
| -----  
  
| Data and other attributes defined here:  
  
|  
| __annotations__ = {'map_type': 'str | None'}  
  
|  
| levels = None  
  
|
```

```
| lookup_table = None
```

```
|
```

```
| map_type = None
```

```
class SizeMapping(SemanticMapping)
```

```
| SizeMapping(plotter, sizes=None, order=None, norm=None)
```

```
|
```

```
| Mapping that sets artist sizes according to data values.
```

```
|
```

```
| Method resolution order:
```

```
|     SizeMapping
```

```
|     SemanticMapping
```

```
|     builtins.object
```

```
|
```

```
| Methods defined here:
```

```
|
```

```
| __init__(self, plotter, sizes=None, order=None, norm=None)
```

```
|     Map the levels of the `size` variable to distinct values.
```

```
|
```

```
|     Parameters
```

```
|     -----
```

```
|     # TODO add generic parameters
```

```
|
```

```
| categorical_mapping(self, data, sizes, order)
```

```
|
```

```
| infer_map_type(self, norm, sizes, var_type)
```

```
|
```

```
| numeric_mapping(self, data, sizes, norm)

|

| -----

| Data and other attributes defined here:

|

| norm = None

|

| -----

| Methods inherited from SemanticMapping:

|

| __call__(self, key, *args, **kwargs)

|     Get the attribute(s) values for the data key.

|

| -----

| Data descriptors inherited from SemanticMapping:

|

| __dict__

|     dictionary for instance variables

|

| __weakref__

|     list of weak references to the object

|

| -----

| Data and other attributes inherited from SemanticMapping:

|

| __annotations__ = {}

|
```

```
| levels = None
```

```
|
```

```
| lookup_table = None
```

```
|
```

```
| map_type = None
```

```
class StyleMapping(SemanticMapping)
```

```
| StyleMapping(plotter, markers=None, dashes=None, order=None)
```

```
|
```

```
| Mapping that sets artist style according to data values.
```

```
|
```

```
| Method resolution order:
```

```
|     StyleMapping
```

```
|     SemanticMapping
```

```
|     builtins.object
```

```
|
```

```
| Methods defined here:
```

```
|
```

```
| __init__(self, plotter, markers=None, dashes=None, order=None)
```

```
|     Map the levels of the `style` variable to distinct values.
```

```
|
```

```
|     Parameters
```

```
|     -----
```

```
|     # TODO add generic parameters
```

```
|
```

```
|     -----
```

```
| Data and other attributes defined here:
```

```
|  
  
| map_type = 'categorical'  
  
|  
  
| -----  
  
| Methods inherited from SemanticMapping:  
  
|  
  
| __call__(self, key, *args, **kwargs)  
  
|     Get the attribute(s) values for the data key.  
  
|  
  
| -----  
  
| Data descriptors inherited from SemanticMapping:  
  
|  
  
| __dict__  
  
|     dictionary for instance variables  
  
|  
  
| __weakref__  
  
|     list of weak references to the object  
  
|  
  
| -----  
  
| Data and other attributes inherited from SemanticMapping:  
  
|  
  
| __annotations__ = {}  
  
|  
  
| levels = None  
  
|  
  
| lookup_table = None
```

```
class VariableType(collections.UserString)

|   VariableType(data)

|

|   Prevent comparisons elsewhere in the library from using the wrong name.

|

|   Errors are simple assertions because users should not be able to trigger

|   them. If that changes, they should be more verbose.

|

|   Method resolution order:

|       VariableType

|       collections.UserString

|       collections.abc.Sequence

|       collections.abc.Reversible

|       collections.abc.Collection

|       collections.abc.Sized

|       collections.abc.Iterable

|       collections.abc.Container

|       builtins.object

|

|   Methods defined here:

|

|   __eq__(self, other)

|       Return self==value.

|

|   __init__(self, data)

|       Initialize self. See help(type(self)) for accurate signature.

|
```

```
| -----  
  
| Data and other attributes defined here:  
  
|  
| __abstractmethods__ = frozenset()  
  
|  
| __hash__ = None  
  
|  
| allowed = ('numeric', 'datetime', 'categorical')  
  
|  
| -----  
  
| Methods inherited from collections.UserString:  
  
|  
| __add__(self, other)  
  
|  
| __complex__(self)  
  
|  
| __contains__(self, char)  
  
|  
| __float__(self)  
  
|  
| __ge__(self, string)  
|     Return self>=value.  
  
|  
| __getitem__(self, index)  
  
|  
| __getnewargs__(self)  
  
|
```

```
| __gt__(self, string)

|     Return self>value.

|

| __int__(self)

|

| __le__(self, string)

|     Return self<=value.

|

|

| __len__(self)

|

|

| __lt__(self, string)

|     Return self<value.

|

|

| __mod__(self, args)

|

|

| __mul__(self, n)

|

|

| __radd__(self, other)

|

|

| __repr__(self)

|     Return repr(self).

|

|

| __rmod__(self, template)

|

|

| __rmul__ = __mul__(self, n)

|

|

| __str__(self)
```



```

|     Return str(self).
|
|
|   capitalize(self)
|
|     # the following methods are defined in alphabetical order:
|
|
|   casefold(self)
|
|
|   center(self, width, *args)
|
|
|   count(self, sub, start=0, end=9223372036854775807)
|
|     S.count(value) -> integer -- return number of occurrences of value
|
|
|   encode(self, encoding='utf-8', errors='strict')
|
|
|   endswith(self, suffix, start=0, end=9223372036854775807)
|
|
|   expandtabs(self, tabsize=8)
|
|
|   find(self, sub, start=0, end=9223372036854775807)
|
|
|   format(self, /, *args, **kwds)
|
|
|   format_map(self, mapping)
|
|
|   index(self, sub, start=0, end=9223372036854775807)
|
|     S.index(value, [start, [stop]]) -> integer -- return first index of value.
|
|     Raises ValueError if the value is not present.

```

|

| Supporting start and stop arguments is optional, but

| recommended.

|

| isalnum(self)

|

| isalpha(self)

|

| isascii(self)

|

| isdecimal(self)

|

| isdigit(self)

|

| isidentifier(self)

|

| islower(self)

|

| isnumeric(self)

|

| isprintable(self)

|

| isspace(self)

|

| istitle(self)

|

| isupper(self)

```
|  
  
| join(self, seq)  
  
|  
  
| ljust(self, width, *args)  
  
|  
  
| lower(self)  
  
|  
  
| lstrip(self, chars=None)  
  
|  
  
| partition(self, sep)  
  
|  
  
| removeprefix(self, prefix, /)  
  
|  
  
| removesuffix(self, suffix, /)  
  
|  
  
| replace(self, old, new, maxsplit=-1)  
  
|  
  
| rfind(self, sub, start=0, end=9223372036854775807)  
  
|  
  
| rindex(self, sub, start=0, end=9223372036854775807)  
  
|  
  
| rjust(self, width, *args)  
  
|  
  
| rpartition(self, sep)  
  
|  
  
| rsplit(self, sep=None, maxsplit=-1)  
  
|
```

```
| rstrip(self, chars=None)

|

|  split(self, sep=None, maxsplit=-1)

|

|  splitlines(self, keepends=False)

|

|  startswith(self, prefix, start=0, end=9223372036854775807)

|

|  strip(self, chars=None)

|

|  swapcase(self)

|

|  title(self)

|

|  translate(self, *args)

|

|  upper(self)

|

|  zfill(self, width)

|  -----

|  Static methods inherited from collections.UserString:

|

|  maketrans(x, y=<unrepresentable>, z=<unrepresentable>, /)

|      Return a translation table usable for str.translate().

|

|      If there is only one argument, it must be a dictionary mapping Unicode
```

| ordinals (integers) or characters to Unicode ordinals, strings or None.

| Character keys will be then converted to ordinals.

| If there are two arguments, they must be strings of equal length, and

| in the resulting dictionary, each character in x will be mapped to the

| character at the same position in y. If there is a third argument, it

| must be a string, whose characters will be mapped to None in the result.

|

| -----

| Data descriptors inherited from collections.UserString:

|

| __dict__

| dictionary for instance variables

|

| __weakref__

| list of weak references to the object

|

| -----

| Methods inherited from collections.abc.Sequence:

|

| __iter__(self)

|

| __reversed__(self)

|

| -----

| Class methods inherited from collections.abc.Reversible:

|

| __subclasshook__(C)

```
|     Abstract classes can override this to customize issubclass().
|
|
|     This is invoked early on by abc.ABCMeta.__subclasscheck__().
|
|     It should return True, False or NotImplemented.  If it returns
|
|     NotImplemented, the normal algorithm is used.  Otherwise, it
|
|     overrides the normal algorithm (and the outcome is cached).
|
|
| -----
|
| Class methods inherited from collections.abc.Iterable:
|
|
| __class_getitem__ = GenericAlias(args, /)
|
|     Represent a PEP 585 generic type
|
|
|     E.g. for t = list[int], t.__origin__ is list and t.__args__ is (int,).
```

```
class VectorPlotter(builtins.object)
```

```
|     VectorPlotter(data=None, variables={})
|
|
|     Base class for objects underlying *plot functions.
|
|
|     Methods defined here:
|
|
|     __init__(self, data=None, variables={})
|
|         Initialize self.  See help(type(self)) for accurate signature.
|
|
|     add_legend_data(self, ax, func, common_kws=None, attrs=None, semantic_kws=None)
|
|         Add labeled artists to represent the different plot semantics.
```

```

|
|  assign_variables(self, data=None, variables={})
|
|      Define plot variables, optionally using lookup from `data`.
|
|
|  iter_data(
|
|      self,
|
|      grouping_vars=None,
|
|      *,
|
|      reverse=False,
|
|      from_comp_data=False,
|
|      by_facet=True,
|
|      allow_empty=False,
|
|      dropna=True
|
|  )
|
|      Generator for getting subsets of data defined by semantic variables.
|
|
|      Also injects "col" and "row" into grouping semantics.
|
|
|      Parameters
|
|      -----
|
|      grouping_vars : string or list of strings
|
|          Semantic variables that define the subsets of data.
|
|      reverse : bool
|
|          If True, reverse the order of iteration.
|
|      from_comp_data : bool
|
|          If True, use self.comp_data rather than self.plot_data
|
|      by_facet : bool

```

```

|         If True, add faceting variables to the set of grouping variables.
|
|         allow_empty : bool
|
|         If True, yield an empty dataframe when no observations exist for
|
|         combinations of grouping variables.
|
|         dropna : bool
|
|         If True, remove rows with missing data.
|
|
|
|     Yields
|
|     -----
|
|     sub_vars : dict
|
|         Keys are semantic names, values are the level of that semantic.
|
|     sub_data : :class:`pandas.DataFrame`
|
|         Subset of ``plot_data`` for this combination of semantic values.
|
|
|
|     map_hue(self, palette=None, order=None, norm=None, saturation=1)
|
|
|
|     map_size(self, sizes=None, order=None, norm=None)
|
|
|
|     map_style(self, markers=None, dashes=None, order=None)
|
|
|
|     scale_categorical(self, axis, order=None, formatter=None)
|
|         Enforce categorical (fixed-scale) rules for the data on given axis.
|
|
|
|     Parameters
|
|     -----
|
|     axis : "x" or "y"
|
|         Axis of the plot to operate on.

```



```

|     order : list
|
|         Order that unique values should appear in.
|
|     formatter : callable
|
|         Function mapping values to a string representation.
|
|
|
|     Returns
|
|     -----
|
|     self
|
|
|     scale_datetime(self, axis, *args, **kwargs)
|
|
|     scale_native(self, axis, *args, **kwargs)
|
|
|     scale_numeric(self, axis, *args, **kwargs)
|
|
|     -----
|
|     Readonly properties defined here:
|
|
|     comp_data
|
|         Dataframe with numeric x and y, after unit conversion and log scaling.
|
|
|     has_xy_data
|
|         Return True at least one of x or y is defined.
|
|
|     var_levels
|
|         Property interface to ordered list of variables levels.
|

```

```
|      Each time it's accessed, it updates the var_levels dictionary with the  
|  
|      list of levels in the current semantic mappers. But it also allows the  
|  
|      dictionary to persist, so it can be used to set levels by a key. This is  
|  
|      used to track the list of col/row levels using an attached FacetGrid  
|  
|      object, but it's kind of messy and ideally fixed by improving the  
|  
|      faceting logic so it interfaces better with the modern approach to  
|  
|      tracking plot variables.
```

```
|  
| -----
```

```
| Data descriptors defined here:
```

```
|  
| __dict__  
|  
|     dictionary for instance variables  
|  
|  
| __weakref__  
|  
|     list of weak references to the object  
|
```

```
| -----
```

```
| Data and other attributes defined here:
```

```
|  
| flat_structure = {'x': '@index', 'y': '@values'}  
|  
|  
| wide_structure = {'hue': '@columns', 'style': '@columns', 'x': '@index...
```

FUNCTIONS

```
categorical_order(vector, order=None)
```

```
    Return a list of unique data values.
```

Determine an ordered list of levels in ``values``.

Parameters

vector : list, array, Categorical, or Series

Vector of "categorical" values

order : list-like, optional

Desired order of category levels to override the order determined
from the ``values`` object.

Returns

order : list

Ordered list of category levels not including null values.

`infer_orient(x=None, y=None, orient=None, require_numeric=True)`

Determine how the plot should be oriented based on the data.

For historical reasons, the convention is to call a plot "horizontally"
or "vertically" oriented based on the axis representing its dependent
variable. Practically, this is used when determining the axis for
numerical aggregation.

Parameters

x, y : Vector data or None

Positional data vectors for the plot.

`orient` : string or None

Specified orientation. If not None, can be "x" or "y", or otherwise must start with "v" or "h".

`require_numeric` : bool

If set, raise when the implied dependent variable is not numeric.

Returns

`orient` : "x" or "y"

Raises

ValueError: When ``orient`` is an unknown string.

TypeError: When dependent variable is not numeric, with ``require_numeric``

`unique_dashes(n)`

Build an arbitrarily long list of unique dash styles for lines.

Parameters

`n` : int

Number of unique dash specs to generate.

Returns

`dashes` : list of strings or tuples

Valid arguments for the ``dashes`` parameter on
:class:`matplotlib.lines.Line2D`. The first spec is a solid
line (``""``), the remainder are sequences of long and short
dashes.

`unique_markers(n)`

Build an arbitrarily long list of unique marker styles for points.

Parameters

`n : int`

Number of unique marker specs to generate.

Returns

`markers : list of string or tuples`

Values for defining :class:`matplotlib.markers.MarkerStyle` objects.

All markers will be filled.

`variable_type(vector, boolean_type='numeric')`

Determine whether a vector contains numeric, categorical, or datetime data.

This function differs from the pandas typing API in two ways:

- Python sequences or object-typed PyData objects are considered numeric if all of their entries are numeric.
- String or mixed-type data are considered categorical even if not

explicitly represented as a `:class:`pandas.api.types.CategoricalDtype``.

Parameters

`vector` : `:func:`pandas.Series``, `:func:`numpy.ndarray``, or Python sequence

Input data to test.

`boolean_type` : 'numeric' or 'categorical'

Type to use for vectors containing only 0s and 1s (and NAs).

Returns

`var_type` : 'numeric', 'categorical', or 'datetime'

Name identifying the type of data in the vector.

DATA

```
QUAL_PALETTES = ['tab10', 'tab20', 'tab20b', 'tab20c', 'Set1', 'Set2', ...
```

FILE

```
c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\_base.py
```

Help on module `seaborn._compat` in `seaborn`:

NAME

```
seaborn._compat
```

FUNCTIONS

`get_colormap(name)`

Handle changes to matplotlib colormap interface in 3.6.

`get_layout_engine(fig: Figure) -> mpl.layout_engine.LayoutEngine | None`

Handle changes to auto layout engine interface in 3.6

`get_legend_handles(legend)`

Handle legendHandles attribute rename.

`groupby_apply_include_groups(val)`

`norm_from_scale(scale, norm)`

Produce a Normalize object given a Scale and min/max domain limits.

`register_colormap(name, cmap)`

Handle changes to matplotlib colormap interface in 3.6.

`set_layout_engine(`

`fig: Figure,`

`engine: Literal['constrained', 'compressed', 'tight', 'none']`

`) -> None`

Handle changes to auto layout engine interface in 3.6

`share_axis(ax0, ax1, which)`

Handle changes to post-hoc axis sharing.

`Literal = typing.Literal`

Special typing form to define literal types (a.k.a. value types).

This form can be used to indicate to type checkers that the corresponding variable or function parameter has a value equivalent to the provided literal (or one of several literals)::

```
def validate_simple(data: Any) -> Literal[True]: # always returns True
    ...
```

```
MODE = Literal['r', 'rb', 'w', 'wb']
```

```
def open_helper(file: str, mode: MODE) -> str:
    ...
```

```
open_helper('/some/path', 'r') # Passes type check
```

```
open_helper('/other/path', 'typo') # Error in type checker
```

`Literal[...]` cannot be subclassed. At runtime, an arbitrary value is allowed as type argument to `Literal[...]`, but type checkers may impose restrictions.

FILE

`c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn\_compat.py`

Help on package `seaborn._core` in `seaborn`:

NAME

seaborn._core

PACKAGE CONTENTS

data
exceptions
groupby
moves
plot
properties
rules
scales
subplots
typing

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn_core__init__.py

Help on module seaborn._docstrings in seaborn:

NAME

seaborn._docstrings

CLASSES

builtins.object

DocstringComponents

```
class DocstringComponents(builtins.object)

| DocstringComponents(comp_dict, strip_whitespace=True)

|

| Methods defined here:

|

| __getattr__(self, attr)

|     Provide dot access to entries for clean raw docstrings.

|

| __init__(self, comp_dict, strip_whitespace=True)

|     Read entries from a dict, optionally stripping outer whitespace.

|

| -----

| Class methods defined here:

|

| from_function_params(func)

|     Use the numpydoc parser to extract components from existing func.

|

| from_nested_components(**kwargs)

|     Add multiple sub-sets of components.

|

| -----

| Data descriptors defined here:

|

| __dict__

|     dictionary for instance variables
```

```
|
|
|  __weakref__
|
|      list of weak references to the object
|
|
|  -----
|
|  Data and other attributes defined here:
|
|
|  regexp = re.compile('\n((\n|.)+)\n\s*', re.MULTILINE)
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn_docstrings.py

Help on module seaborn._statistics in seaborn:

NAME

seaborn._statistics - Statistical transformations for visualization.

DESCRIPTION

This module is currently private, but is being written to eventually form part of the public API.

The classes should behave roughly in the style of scikit-learn.

- All data-independent parameters should be passed to the class constructor.
- Each class should implement a default transformation that is exposed through `__call__`. These are currently written for vector arguments, but I think

consuming a whole `plot\_data` DataFrame and return it with transformed

variables would make more sense.

- Some class have data-dependent preprocessing that should be cached and used multiple times (think defining histogram bins off all data and then counting observations within each bin multiple times per data subsets). These currently have unique names, but it would be good to have a common name. Not quite `fit`, but something similar.
- Alternatively, the transform interface could take some information about grouping variables and do a groupby internally.
- Some classes should define alternate transforms that might make the most sense with a different function. For example, KDE usually evaluates the distribution on a regular grid, but it would be useful for it to transform at the actual datapoints. Then again, this could be controlled by a parameter at the time of class instantiation.

CLASSES

builtins.object

ECDF

EstimateAggregator

Histogram

KDE

LetterValues

WeightedAggregator

```
class ECDF(builtins.object)
```

```
| ECDF(stat='proportion', complementary=False)
```

```
|
```

```

| Univariate empirical cumulative distribution estimator.
|
| Methods defined here:
|
| __call__(self, x1, x2=None, weights=None)
|
|     Return proportion or count of observations below each sorted datapoint.
|
|
| __init__(self, stat='proportion', complementary=False)
|
|     Initialize the class with its parameters
|
|
|     Parameters
|
|     -----
|
|     stat : {"proportion", "percent", "count"}
|
|         Distribution statistic to compute.
|
|     complementary : bool
|
|         If True, use the complementary CDF (1 - CDF)
|
|
|     -----
|
| Data descriptors defined here:
|
|
| __dict__
|
|     dictionary for instance variables
|
|
| __weakref__
|
|     list of weak references to the object

```

```

class EstimateAggregator(builtins.object)

```

```

| EstimateAggregator(estimator, errorbar=None, **boot_kws)
|
|
| Methods defined here:
|
|
| __call__(self, data, var)
|
|     Aggregate over `var` column of `data` with estimate and error interval.
|
|
| __init__(self, estimator, errorbar=None, **boot_kws)
|
|     Data aggregator that produces an estimate and error bar interval.
|
|
|     Parameters
|
|     -----
|
|     estimator : callable or string
|
|         Function (or method name) that maps a vector to a scalar.
|
|     errorbar : string, (string, number) tuple, or callable
|
|         Name of errorbar method (either "ci", "pi", "se", or "sd"), or a tuple
|         with a method name and a level parameter, or a function that maps from a
|         vector to a (min, max) interval, or None to hide errorbar. See the
|
|         :doc:`errorbar tutorial </tutorial/errorBars>` for more information.
|
|     boot_kws
|
|         Additional keywords are passed to bootstrap when error_method is "ci".
|
|
|     -----
|
| Data descriptors defined here:
|
|
| __dict__
|
|     dictionary for instance variables

```

```
|  
  
|  __weakref__  
  
|      list of weak references to the object
```

```
class Histogram(builtins.object)
```

```
|  Histogram(  
  
|      stat='count',  
  
|      bins='auto',  
  
|      binwidth=None,  
  
|      binrange=None,  
  
|      discrete=False,  
  
|      cumulative=False  
  
|  )  
  
|  
  
|  Univariate and bivariate histogram estimator.  
  
|  
  
|  Methods defined here:  
  
|  
  
|  __call__(self, x1, x2=None, weights=None)  
  
|      Count the occurrences in each bin, maybe normalize.  
  
|  
  
|  __init__(  
  
|      self,  
  
|      stat='count',  
  
|      bins='auto',  
  
|      binwidth=None,  
  
|      binrange=None,
```

```

|     discrete=False,
|
|     cumulative=False
|
| )
|
| Initialize the estimator with its parameters.
|
|
| Parameters
|
| -----
|
| stat : str
|
|     Aggregate statistic to compute in each bin.
|
|
|     - `count`: show the number of observations in each bin
|
|     - `frequency`: show the number of observations divided by the bin width
|
|     - `probability` or `proportion`: normalize such that bar heights sum to 1
|
|     - `percent`: normalize such that bar heights sum to 100
|
|     - `density`: normalize such that the total area of the histogram equals 1
|
|
| bins : str, number, vector, or a pair of such values
|
|     Generic bin parameter that can be the name of a reference rule,
|
|     the number of bins, or the breaks of the bins.
|
|     Passed to :func:`numpy.histogram_bin_edges`.
|
| binwidth : number or pair of numbers
|
|     Width of each bin, overrides ``bins`` but can be used with
|
|     ``binrange``.
|
| binrange : pair of numbers or a pair of pairs
|
|     Lowest and highest value for bin edges; can be used either
|
|     with ``bins`` or ``binwidth``. Defaults to data extremes.
|
| discrete : bool or pair of bools

```



```

|         If True, set ``binwidth`` and ``binrange`` such that bin
|
|         edges cover integer values in the dataset.
|
|         cumulative : bool
|
|         If True, return the cumulative statistic.
|
|
|     define_bin_params(self, x1, x2=None, weights=None, cache=True)
|
|         Given data, return numpy.histogram parameters to define bins.
|
|
|     -----
|
|     Data descriptors defined here:
|
|
|     __dict__
|
|         dictionary for instance variables
|
|
|     __weakref__
|
|         list of weak references to the object

```

```

class KDE(builtins.object)

```

```

|     KDE(
|
|         *,
|
|         bw_method=None,
|
|         bw_adjust=1,
|
|         gridsize=200,
|
|         cut=3,
|
|         clip=None,
|
|         cumulative=False
|
|     )

```

```

|
|  Univariate and bivariate kernel density estimator.
|
|  Methods defined here:
|
|  __call__(self, x1, x2=None, weights=None)
|
|      Fit and evaluate on univariate or bivariate data.
|
|  __init__(
|
|      self,
|
|      *,
|
|      bw_method=None,
|
|      bw_adjust=1,
|
|      gridsize=200,
|
|      cut=3,
|
|      clip=None,
|
|      cumulative=False
|
|  )
|
|      Initialize the estimator with its parameters.
|
|
|      Parameters
|
|      -----
|
|      bw_method : string, scalar, or callable, optional
|
|          Method for determining the smoothing bandwidth to use; passed to
|
|          :class:`scipy.stats.gaussian_kde`.
|
|      bw_adjust : number, optional
|
|          Factor that multiplicatively scales the value chosen using

```

```

|         ``bw_method``. Increasing will make the curve smoother. See Notes.

|         gridsize : int, optional
|
|             Number of points on each dimension of the evaluation grid.

|         cut : number, optional
|
|             Factor, multiplied by the smoothing bandwidth, that determines how
|             far the evaluation grid extends past the extreme datapoints. When
|             set to 0, truncate the curve at the data limits.

|         clip : pair of numbers or None, or a pair of such pairs
|
|             Do not evaluate the density outside of these limits.

|         cumulative : bool, optional
|
|             If True, estimate a cumulative distribution function. Requires scipy.

|
|
|         define_support(self, x1, x2=None, weights=None, cache=True)
|
|             Create the evaluation grid for a given data set.
|
|
|         -----
|
|         Data descriptors defined here:
|
|
|         __dict__
|
|             dictionary for instance variables
|
|
|         __weakref__
|
|             list of weak references to the object

```

```

class LetterValues(builtins.object)

```

```

|     LetterValues(k_depth, outlier_prop, trust_alpha)
|

```

```

| Methods defined here:
|
|
|  __call__(self, x)
|
|      Evaluate the letter values.
|
|
|  __init__(self, k_depth, outlier_prop, trust_alpha)
|
|      Compute percentiles of a distribution using various tail stopping rules.
|
|
|      Parameters
|
|      -----
|
|      k_depth: "tukey", "proportion", "trustworthy", or "full"
|
|          Stopping rule for choosing tail percentiled to show:
|
|
|          - tukey: Show a similar number of outliers as in a conventional boxplot.
|
|          - proportion: Show approximately `outlier_prop` outliers.
|
|          - trust_alpha: Use `trust_alpha` level for most extreme tail percentile.
|
|
|      outlier_prop: float
|
|          Parameter for `k_depth="proportion"` setting the expected outlier rate.
|
|      trust_alpha: float
|
|          Parameter for `k_depth="trustworthy"` setting the confidence threshold.
|
|
|      Notes
|
|      -----
|
|      Based on the proposal in this paper:
|
|      https://vita.had.co.nz/papers/letter-value-plot.pdf
|

```

| -----

| Data descriptors defined here:

|

| `__dict__`

| dictionary for instance variables

|

| `__weakref__`

| list of weak references to the object

class WeightedAggregator(builtins.object)

| `WeightedAggregator(estimator, errorbar=None, **boot_kws)`

|

| Methods defined here:

|

| `__call__(self, data, var)`

| Aggregate over `var` column of `data` with estimate and error interval.

|

| `__init__(self, estimator, errorbar=None, **boot_kws)`

| Data aggregator that produces a weighted estimate and error bar interval.

|

| Parameters

| -----

| `estimator : string`

| Function (or method name) that maps a vector to a scalar. Currently

| supports only "mean".

| `errorbar : string or (string, number) tuple`

| Name of errorbar method or a tuple with a method name and a level parameter.

```
|           Currently the only supported method is "ci".

|   boot_kws

|           Additional keywords are passed to bootstrap when error_method is "ci".

|

|   -----

|   Data descriptors defined here:

|

|   __dict__

|           dictionary for instance variables

|

|   __weakref__

|           list of weak references to the object
```

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn_statistics.py

Help on package seaborn._stats in seaborn:

NAME

seaborn._stats

PACKAGE CONTENTS

aggregation

base

counting

density

order

regression

FILE

c:\users\lenovo\appdata\local\programs\python\python314\lib\site-packages\seaborn_stats__init__.py